

Assessing output reliability and similarity of large language models in software development: A comparative case study approach

Dae-Kyoo Kim ^a,*, Hua Ming ^b

^a Computer Science and Engineering, Oakland University, 115 Library Dr., Rochester, 48309, MI, USA

^b Computer Science, University of Michigan, 303 E. Kearsley Street, Flint, 48502, MI, USA

ARTICLE INFO

Dataset link: <https://osf.io/mc96h/>

Keywords:

AI model

LLM

Prompt

Reliability

Similarity

Software development

ABSTRACT

Context: Generative large language models (LLMs) are increasingly used across various activities in software development, offering significant potential to enhance productivity. However, there is a lack of systematic study examining the reliability and similarity of the outputs from these models.

Objective: This work presents a comparative analysis of the reliability – defined as the consistency and correctness of software artifacts – and similarity of LLM outputs in software development.

Method: To accomplish the objective, we introduce a structured approach for assessing the reliability and similarity of outputs from five prominent LLMs – ChatGPT, Claude, Copilot, Gemini, and Meta – and apply it within two case studies focused on developing a food order and delivery system and a smart wallet system.

Results: The study found that the overall output reliability of the models is rated at 0.82 with Claude outperforming other models at 0.92, followed by ChatGPT at 0.90, Copilot at 0.80, Meta at 0.75, and Gemini at 0.71. The models demonstrated an overall 57% similarity and 43% variability in their outputs, highlighting the uniqueness of models.

Conclusions: While overall, LLMs exhibit decent reliability in their outputs with varying degrees, they still require human oversight and review of their outputs before implementation. LLMs present unique characteristics that practitioners should consider before adoption.

1. Introduction

Generative large language models (LLMs) are increasingly utilized in software development, offering great potential to improve productivity. These models assist in various tasks across different phases, such as gathering requirements during the analysis phase, developing designs in the modeling phase, generating code and fixing errors in the implementation phase, and creating test cases in the testing phase. However, LLMs are fallible and often produce erroneous outputs [1–3], which can have substantial impacts on a project, especially when such outputs are adopted in the early phases of development due to the interconnected nature of the software development process. Thus, it is essential to evaluate the reliability of the outputs from LLMs before integrating them into practice, which helps to mitigate potential risks throughout development. Moreover, as various LLMs claim superiority and uniqueness, and with limited transparency on how these models were trained and their similarities and differences, it is essential to understand these aspects to make informed decisions about adopting a particular model in practice.

There is growing research on the applications of LLMs in diverse software engineering tasks. Most research has focused on producing programming solutions, particularly using Copilot [4] with different interests such as problem-solving proficiency [1], program synthesis [5], translating from natural language descriptions [2], productivity improvement [6], pedagogical implications [7], correctness and understandability [8], and code validity [9]. Some studies have explored early development phases such as requirements analysis and design modeling [3]. There is also a comparative study examining the proficiency of ChatGPT [10] and Bard (a predecessor of Gemini [11]) in software development tasks [12]. While the existing work provides valuable insights into the capabilities of LLMs, there remains a significant gap in systematic quantification and comparison of the reliability and similarity of LLM outputs throughout the development process.

To address the gap, we present a study on evaluating the reliability and similarity of LLMs for their outputs in software development. We introduce a structured approach with a set of metrics to assess the reliability and similarity of software artifacts produced by LLMs at different levels of the development process, designed for comparative

* Corresponding author.

E-mail address: kim2@oakland.edu (D.-K. Kim).

<https://doi.org/10.1016/j.infsof.2025.107787>

Received 3 August 2024; Received in revised form 18 April 2025; Accepted 12 May 2025

Available online 29 May 2025

0950-5849/© 2025 Elsevier B.V. All rights reserved, including those for text and data mining, AI training, and similar technologies.

analysis rather than optimization of individual LLMs. Applying the approach, we evaluate five prominent LLMs – ChatGPT, Claude [13], Copilot, Gemini, and Meta [14] for the reliability and similarity of their outputs through two case studies on the development of a food order and delivery system and a smart wallet system. The reliability analysis enables understanding strengths and weaknesses of LLMs at the activity level, the phase level, and the entire process level, while the similarity analysis offers insights into how similar their outputs are and helps us better understand the uniqueness of individual models. The evaluation indicates that in terms of reliability, Claude and ChatGPT perform better with Claude achieving the highest reliability score of 0.92 out of 1, and ChatGPT scoring 0.90, while Copilot, Meta, and Gemini exhibit moderate performance with reliability scores of 0.80, 0.75, and 0.71, respectively, contributing to an overall score of 0.81. With respect to similarity, the models exhibit a similar degree of similarity with an overall similarity score of 2.87 out of 5, indicating that they are 57% similar and 43% different. This difference presents the uniqueness of individual models. These findings suggest that LLMs still require human oversight and review of their outputs before implementation, and they present unique characteristics to be considered before adoption. This study aims to address the following research questions:

- RQ1 *How can the reliability and similarity of software artifacts produced by LLMs be measured?* This question seeks to establish an approach for evaluating the reliability of LLM outputs and the similarity in results among different models in performing the same task.
- RQ2 *How reliable are the software artifacts produced by LLMs?* This question aims to assess and compare the reliability of the outputs from individual models, providing a benchmark for developers to consider when choosing LLMs for their needs.
- RQ3 *How similar are the software artifacts produced by LLMs to each other?* This question aims to evaluate the similarity and variability among the outputs of individual LLMs, offering insights into their unique characteristics for developers to consider when adopting them in practice.

The remainder of the paper is structured as follows. Section 2 reviews existing work on the use of LLMs in software engineering. Section 3 introduces the approach for measuring the reliability and similarity of software artifacts produced by LLMs. Section 4 presents the evaluation of LLMs in terms of reliability and similarity through case studies. Section 5 discusses the findings in relation to the research questions posed earlier and provides qualitative insights. Section 6 addresses threats to validity. Finally, Section 7 concludes the paper with a discussion on future work.

2. Related work

Recent work on LLMs in software development can be broadly categorized into (i) empirical evaluations across development phases, (ii) code generation capabilities, (iii) requirements and design aspects, and (iv) systematic approaches for LLM integration.

2.1. Software development evaluations

Several studies have evaluated LLMs across multiple phases of software development, most closely relating to this work. Kim et al. [3] examined ChatGPT's capabilities through a tour reservation system case study, finding it capable of handling over 90% of coding tasks but noting issues with traceability and consistency. Building on this, Kim [12] compared ChatGPT and Bard using the same case study, revealing their complementary strengths in requirements analysis and user experience. While these studies provide valuable insights into individual models' capabilities, they lack the systematic quantitative evaluation framework that we introduce in this work for comparing multiple LLMs.

Sridhara et al. [15] evaluated ChatGPT across fifteen distinct software engineering tasks (e.g., code summarization, bug repair, conflict resolution), showing varied effectiveness across tasks. Champa et al. [16] analyzed developer interactions with ChatGPT using the DevGPT dataset [17] of 2865 conversations, finding that prompt quality has minimal impact on assistance effectiveness. Nguyen et al. [18] identified 78 research questions across 11 areas in generative AI (GenAI) software engineering, highlighting gaps in requirements engineering and design research. Rahmaniari [19] examined ChatGPT's role in enhancing software engineering processes, focusing on its impact on coding efficiency, contextual understanding, and economic benefits, while underscoring the need for rigorous standards and ethical considerations in its deployment. Waseem et al. [20] investigated ChatGPT's effectiveness in supporting software development tasks in educational contexts, noting its benefits for student understanding while cautioning against over-dependency that might affect critical thinking skills. Wang et al. [21] evaluated LLM system performance through analysis of real-world serving workloads, focusing on metrics like request concurrency and failure rates.

Felizardo et al. [22] evaluated ChatGPT's effectiveness in systematic literature reviews, reporting accuracy rates of 75.3-86.1% but emphasizing the need for human verification. This expanding body of research demonstrates both the potential and limitations of LLMs in early development phases, consistently emphasizing the necessity of human oversight. Fan et al. [23] surveyed LLM applications across various software engineering aspects, highlighting both opportunities (e.g., improved productivity and quality) and reliability challenges (e.g., hallucinations). Wermelinger [7] examined Copilot's pedagogical implications, raising concerns about academic integrity while acknowledging its potential in educational settings.

These studies demonstrate LLMs' potential in software development tasks, while highlighting varying effectiveness across applications. While existing research provides valuable insights into individual model capabilities, there remains a gap in systematic quantification and comparison of the quality of LLM outputs throughout the development process.

2.2. Code generation and quality assessment

A significant body of research focuses on code generation capabilities, particularly regarding Copilot. Dakhel et al. [1] and Mastropaolo et al. [2] evaluated Copilot's reliability in algorithmic problem-solving and code generation, finding issues with reproducibility and semantic equivalence. Peng et al. [6] demonstrated Copilot's impact on productivity, while Nguyen and Nadi [8] and Yetistiren [9] assessed code quality across different languages. Zhong and Wang [24] specifically examined API misuse in LLM-generated code. Sobania et al. [5] compared Copilot with genetic programming, finding Copilot more practical for daily use. Pudari and Ernst [25] analyzed Copilot's adherence to language-specific idioms, noting its strength in syntactic correctness but limitations in implementing programming conventions and design patterns. Zhang et al. [26] evaluated ChatGPT's bug-fixing capabilities through benchmark testing, demonstrating success in automated program repair while raising concerns about training data leakage. Nascimento et al. [27] compared ChatGPT against human programmers on LeetCode, finding inconsistent performance in complex problem-solving scenarios.

These studies reveal a complex trade-off between development efficiency and code reliability, where LLMs can significantly accelerate coding tasks (e.g., showing up to 55.8% faster development times [6]) but may introduce subtle defects (e.g., API misuse rates of up to 27% in certain languages [8]). Their findings emphasize the importance of robust validation mechanisms and developer expertise in effectively utilizing LLM-generated code.

2.3. Requirements and design aspects

Research on LLMs in requirements and design phases has gained increasing attention. Marques et al. [28] evaluated ChatGPT's role in requirements engineering, highlighting its effectiveness in enhancing stakeholder communication and requirement documentation while noting challenges with bias and accuracy in requirements elicitation. Ahmad et al. [29] explored ChatGPT's integration in software architecting, developing a collaborative framework where ChatGPT assists architects in requirement articulation, pattern suggestion, and design automation, demonstrated through a microservices architecture case study. Bera et al. [30] assessed ChatGPT's capability in agile practices by analyzing Twitter sentiments on its trustworthiness and empirically evaluating its performance in agile-related queries, finding it effective in concept summarization but requiring careful integration into professional teams.

Their findings consistently emphasize that while LLMs demonstrate promising capabilities in requirements and design tasks (e.g., stakeholder communication, architectural decision-making, agile practice guidance), significant challenges remain (e.g., output inconsistency, ethical considerations, reliability concerns). The studies highlight that human expertise remains essential for validating and refining LLM outputs in these critical early development phases.

2.4. Systematic approaches and frameworks

Several researchers have proposed systematic approaches for LLM integration in software development. Alshahwan et al. [31] introduced an approach for improving code, employing generate-and-test methods to ensure reliable code improvements. Rajbhoj et al. [32] explored GenAI integration across development phases, emphasizing prompt-driven development that enables domain experts to interact with models through natural language. Solohubov et al. [33] demonstrated significant efficiency gains through systematic integration of ChatGPT and Copilot, showing how these tools can enhance code generation and error correction while reducing routine tasks. Ozpolat et al. [34] examined the systematic deployment of ChatGPT in development processes, emphasizing the need for careful oversight particularly in code generation and testing tasks to ensure secure and responsible AI utilization.

From an interaction perspective, Hassan et al. [35] proposed developing more collaborative AI pair programmers to address current limitations (e.g., task-focused nature, lack of project context awareness, inability to maintain ongoing dialogues about design decisions) in existing AI models. White et al. [36] presented prompt patterns for enhanced LLM interactions, though without empirical validation.

These integration approaches represent a shift towards more structured integration of LLMs in development workflows, emphasizing the need for systematic validation and context-aware interactions. Their approaches suggest that successful LLM integration requires not just improved models, but also well-defined methodologies for their deployment and validation in real-world development environments.

2.5. Summary

This work builds upon and extends these existing studies in several key ways – (i) This work introduces metrics to quantitatively assess the reliability and similarity of outputs generated by multiple LLMs, addressing a gap in systematic evaluation methods; (ii) This work provides comprehensive evaluation across all development phases rather than focusing on specific aspects; (iii) This work compares multiple contemporary LLMs (ChatGPT, Claude, Copilot, Gemini, and Meta) using the same evaluation criteria; (iv) This work demonstrates the approach through two distinct case studies, validating its applicability across different domains.

3. Quantifying LLM reliability and similarity

In this section, we introduce a method for quantifying reliability and similarity of software artifacts produced by LLMs. Fig. 1 illustrates the evaluation workflow, which starts with system descriptions that serve as inputs to the development process. The process follows a structured approach based on object-oriented development practices [37] to evaluate artifacts at different hierarchical levels – activity, phase, and overall process. An activity is a distinct task that produces specific software artifacts, focusing on a particular aspect of software development and resulting in outputs that contribute to the overall project. A phase is a major stage in software development that comprises a set of related activities, grouping together activities that contribute to a common segment of the development lifecycle. We analyze four phases: (i) Requirements Analysis, which includes activities related to the gathering and analysis of software requirements, (ii) Design Modeling, which involves activities focusing on creating software designs, (iii) Implementation, which focuses on the actual coding of the software, and (iv) Testing, which verifies and validates the software meeting the requirements. The process encompasses the entire software development lifecycle, integrating all phases and activities from initial requirements gathering to final testing. The process represents a complete cycle of software development, with each phase contributing to the progression towards a finished software product. LLMs interact with this process through prompts corresponding to each activity. The evaluation metrics in this work are defined at each level of this hierarchy – metrics at the activity level assess individual task outputs, phase-level metrics aggregate activity measures within each development stage, and process-level metrics provide overall assessment across the complete development lifecycle. For each metric category (reliability and similarity), we maintain this hierarchical structure to evaluate outputs from fine-grained activities to the complete process. This multi-level evaluation framework allows us to systematically assess the quality of generated artifacts while accounting for their relationships and dependencies across different development activities and phases.

3.1. Reliability

In this study, reliability refers to the consistent production of valid software artifacts by LLMs throughout the development process. This differs from traditional software reliability metrics that focus on runtime behavior [38]. At the activity level, reliability is assessed based on the validity and consistency of the artifacts produced by individual models for activities, each focusing on a specific task. Validity pertains to the degree to which a software artifact conforms to the syntactic rules and semantic intentions of the notation used for its representation. Consistency examines how well the software artifact aligns with artifacts produced in previous related activities. At the phase level, reliability is determined by aggregating the reliability measures from individual activities within that phase. Finally, at the process level, reliability is gauged by combining the reliability assessments of each phase to reflect the overall reliability of the model throughout the development process.

Validity. The validity of artifacts produced by a model in an activity is assessed based on the number of elements in the output that violate either the structural rules or the intended meaning of the notation used for their representation. Structural rules refer to the syntactical requirements and notational conventions that must be followed for a given artifact type. These are objective, verifiable properties. Intended meaning concerns the semantic correctness and logical soundness of the content within the properly structured notation. This ensures that the artifact fulfills its intended purpose in software development. Let I represent the total number of produced elements in an activity, and I_{err} represent the number of elements that fail to adhere to syntactic or semantic correctness. The validity V is then calculated as follows:

$$V = 1 - \frac{I_{err}}{I} \quad (1)$$

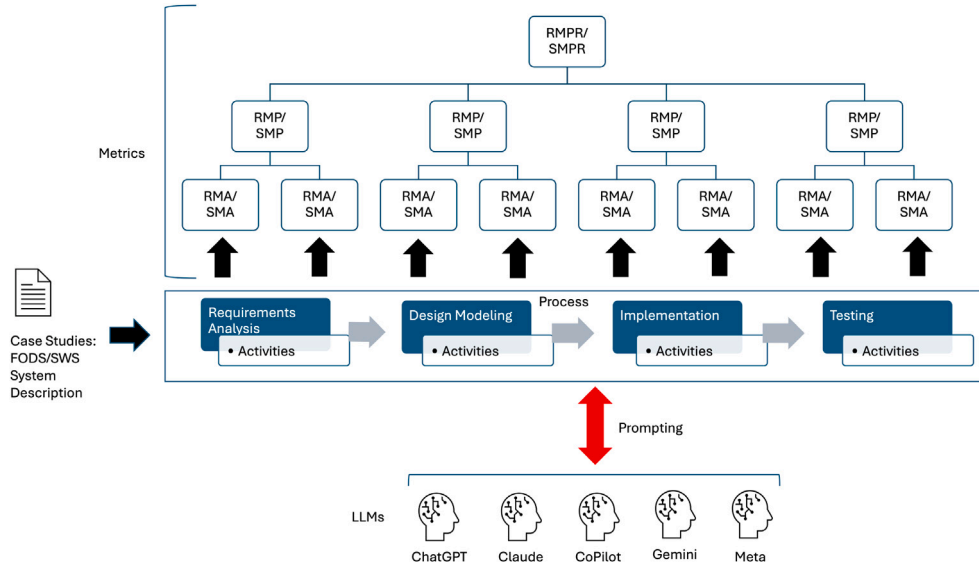


Fig. 1. Assessing reliability and similarity of LLM output via case studies.

This formula gives a validity score on a scale from 0 to 1 with higher scores indicating greater validity.

Consistency. Consistency refers to the degree to which elements in a current artifact align with related elements in previously produced artifacts, ensuring traceability and coherence across the development process. Consistency is evaluated by identifying elements that do not align with previously produced artifacts. Let S represent the total number of unique elements expected to be aligned across relevant artifacts, and S_{mis} represent the number of elements that do not align properly with relevant artifacts. Then, consistency C is calculated as follows:

$$C = 1 - \frac{S_{mis}}{S} \quad (2)$$

This formula provides a consistency score on a scale from 0 to 1 with higher scores indicating greater consistency.

Reliability Metric for Activity (RMA). The reliability of an activity is measured based on the validity and consistency scores of its artifacts. Let w_v and w_c represent the weights assigned to validity and consistency, respectively. These weights are used to prioritize validity and consistency according to their significance within the evaluation context. In this study, an equal weight is given to validity and consistency. The reliability of an activity is calculated as follows:

$$RMA = \frac{V \cdot w_v + C \cdot w_c}{w_v + w_c} \quad (3)$$

Reliability Metric for Phase (RMP). The reliability of a phase is calculated by aggregating the reliability scores of the activities within that phase, taking into account the respective importance of individual activities. Let RMA_i represent the reliability measure for activity i and w_i represent the weight assigned to activity i , reflecting the relative importance of the activity. Then, the reliability of a phase is calculated as follows:

$$RMP = \frac{\sum(RMA_i \cdot w_i)}{\sum w_i} \quad (4)$$

Reliability Metric for Process (RMPPR). The reliability of the overall process is measured by aggregating the reliability of involved phases with consideration of their respective importance. Let RMP_j represent the reliability metric for phase j and w_j represent the weight of phase j , reflecting the relative importance of the phase. Then, the reliability

of a process is calculated as follows:

$$RMPPR = \frac{\sum(RMP_j \cdot w_j)}{\sum w_j} \quad (5)$$

The following outlines the application of reliability metrics described above.

1. Measure the validity and consistency of artifacts produced by individual activities.
2. Measure the RMA for each activity using predefined weights for validity and consistency.
3. Measure the RMP for each phase by averaging RMAs of the activities involved in that phase with predefined weights for each activity.
4. Measure the RMPPR for the overall process by averaging RMPs of all phases involved using predefined weights for each phase.

3.2. Similarity

Similar to measuring reliability, we assess the similarity of LLMs at the activity, phase, and process levels.

Similarity Metric for Activity (SMA). The similarity of models in an activity is assessed by comparing their outputs produced in the same activity. Specifically, the following steps are undertaken.

1. Identify common and unique properties of the outputs from LLMs. Define CM as the number of common properties, and UQ_m as the number of unique properties for model m . Then, the total number of properties for model m is calculated as $Total_m = CM + UQ_m$.
2. Calculate the similarity ratio for each pair of models. For a pair consisting of models a and b , let $Avg = (Total_a + Total_b)/2$. Then, the similarity ratio between a and b is calculated as follows.

$$SR = \frac{CM}{Avg} \quad (6)$$

3. Construct a similarity table by mapping the similarity ratios for each pair of models to the following 0-5 scale.

- 0 [completely different]: $SR < 0.1$ denoting no or negligible overlap.
- 1 [a little similar]: $0.1 \leq SR < 0.3$ denoting very few shared.

- 2 [somewhat similar]: $0.3 \leq SR < 0.5$ denoting some overlap, but significant unique.
- 3 [similar]: $0.5 \leq SR < 0.7$ denoting many shared, but several unique aspects.
- 4 [very similar]: $0.7 \leq SR < 0.9$ denoting most match with minor differences.
- 5 [nearly identical]: $SR \geq 0.9$ denoting nearly all the same.

4. Compute the similarity of an activity as the average similarity from the table. Let n represent the number of pairs of models, and s_i represent the similarity value for pair i where $1 \leq i \leq n$. Then, the similarity of an activity is computed as follows.

$$SMA = \frac{1}{n} \sum_{i=1}^n s_i \quad (7)$$

Similarity Metric for Phase (SMP). The similarity of a phase is determined by averaging the similarities of the activities within that phase. Let n represent the number of activities in the phase, and SMA_i represent the similarity score of activity i where $1 \leq i \leq n$. Then, the similarity of a phase is calculated as follows.

$$SMP = \frac{1}{n} \sum_{i=1}^n SMA_i \quad (8)$$

Similarity Metric for Process (SMPR). The similarity of the overall process is calculated by averaging the similarities of all phases involved in the process, weighted by the number of activities in each phase to reflect their respective influence. Let n represent the total number of activities across all phases, p represent the number of phases, and n_i represent the number of activities in phase i . Let SMP_i represent the similarity score of phase p . Then, the similarity of the process is calculated as follows.

$$SMPR = \frac{\sum_{i=1}^p (n_i \times SMP_i)}{n} \quad (9)$$

The following outlines the application of similarity metrics described above.

1. Measure the SMA for each activity.
2. Measure the SMP for each phase by averaging SMAs of the activities involved in that phase.
3. Measure the SMPR for the overall process by averaging SMAs of all phases involved using the weights for each phase.

4. Experiments: Case studies

In this section, we evaluate the reliability and similarity of LLMs using the metrics discussed in the previous section through two case studies on the development of a food order and delivery system (FODS) and a smart wallet system (SWS). We consider five popular LLMs – ChatGPT (ChatGPT-4o, released in May 2024), Claude (Claude 3.5 Sonnet, released in June 2024), Copilot (Microsoft 365 Copilot, released in November 2023), Gemini (Gemini 1.5, released in February 2024), and Meta (Llama 3.1, released in July 2024). For a fair comparison, we used the non-paid versions of all models. The free versions of these models typically come with limitations compared to their paid counterparts, such as usage caps, slower response times, and restricted access to advanced features, including the ability to process files or images. Usage caps vary. For instance, Claude implements a time-based cap requiring several hours' wait before continued use, while Copilot enforces a cap of five prompts per session in non-Microsoft browsers, though switching to Edge allows up to 30 prompts. In addition, some models, such as Claude and Meta, have output length limitations, necessitating subsequent prompts to instruct the model to continue its previous response.

Table 1 and Table 2 describe a series of development activities organized into columns showing the development phase, activity ID,

activity description, validity criteria, and consistency criteria, where each activity serves as a prompt. For example, activity P2 prompts the model to “Create a use case diagram for the system.” Prompts are crafted to be clear, concise, and task-specific, with minimal wording to reduce ambiguity and avoid introducing bias. The intent was to allow each model to independently interpret and respond to the same instruction without being influenced by overly prescriptive language or domain-specific cues. The output's validity is assessed by checking if it includes necessary components like system boundary, actors, use cases, and relationships between actors and use cases, while consistency is verified against the functional requirements identified in P1. This set of prompts was executed on each model once in sequence, without repeating or branching into multiple conversation threads. This single-thread execution approach was chosen to evaluate baseline, first-attempt performance to (i) establish a controlled comparison framework across models, (ii) assess the innate capabilities of each model without refinement iterations, and (iii) provide insights into the minimum expected performance level. Given this, reliability in this work is considered specifically within the context of single-threaded execution.

Fig. 2 presents the description of two systems – a food order and delivery system (FODS) and a smart wallet system (SWS) – which are used in the case studies and serve as the basis for the development process. The choice of the systems is grounded in their suitability for object-oriented development, a common paradigm in software engineering. These systems, while both suitable for illustrating the divide-and-conquer principle in object-oriented methodology, are fundamentally different in their context and functionality. The food order and delivery system primarily assesses the models' ability to handle traditional business-oriented workflows through interactions between multiple entities (e.g., customers, delivery personnel, and restaurants), each with distinct roles and behaviors, while the smart wallet system evaluates the models' handling of non-traditional system-oriented processes involving novel technologies such as smart contracts and blockchain networks, focusing on digital assets (e.g., cryptocurrencies), authentication, and secure transactions.

Based on the descriptions, the series of activities in Table 1 and Table 2 is carried out across development phases to develop the systems. The Requirements Analysis phase includes activities P1 to P5. P1 focuses on identifying functional and non-functional requirements from the system description with validity assessed through proper categorization into functional and non-functional requirements, and consistency verified against capabilities described in the system description. P2 involves developing a use case model where validity ensures proper notations for system boundary, actors, use cases, and relationships, and consistency is verified against the functional requirements identified in P1. P3 develops detailed use case specifications with validity requiring well-structured descriptions containing pre/post conditions, main success scenario, and alternative scenarios for each use case, and consistency checked against the actors and use cases identified in P2. P4 aims to create a domain model where validity involves proper notation for classes, attributes, relationships, and multiplicities, and consistency is maintained by covering concepts described in the use case specifications in P3. P5 identifies system operations with validity focusing on proper specification of system operations with parameters and return types, and consistency with operations described in the use case specifications in P3.

The Design Modeling phase encompasses activities P6 and P7. P6 involves developing design sequence diagrams with validity ensured by proper notation for sequence diagrams including objects, lifelines, messages, and returns, and consistency with the system operations identified in P5. P7 develops design class diagrams with validity focusing on proper notation for classes with typed attributes and operations, relationships, multiplicities, and navigabilities, and ensuring consistency with the participants and operations in the design sequence diagrams in P6. Design class diagrams are distinguished from domain

Table 1
Development activities: P1–P5.

Phase	Activities (Prompts)	Validity		Consistency	
		Structural rules	Intended meaning	Fit	Alignment criteria
RA	P1: Identify functional and non-functional requirements from the system description.	1. Functional requirements categorized by features. 2. Non-functional requirements categorized by quality attributes. 3. Each requirement uniquely identified.	1. Requirements are atomic. 2. Requirements address system description needs. 3. Clear distinction between functional and non-functional aspects.	SD	1. Each functional requirement maps to system description features. 2. Non-functional requirements reflect system constraints. 3. No requirements contradict system description.
	P2: Create a use case model for the system.	1. System boundary present. 2. Actors connected to at least one use case. 3. Proper relationship notation. 4. Unique identifiers for use cases.	1. Use cases represent complete units of functionality. 2. Actors represent distinct external entities. 3. Relationships logically connect related functionalities. 4. Use cases collectively cover system functionality.	P1	1. Each use case traces to at least one functional requirement. 2. Actor interactions reflect system boundaries defined in requirements. 3. Use case relationships align with functional dependencies in requirements. 4. All functional requirements are covered by use cases.
	P3: Create use case specifications for identified use cases.	1. Required sections present (primary actor, pre/post conditions, main flow, alternative flows). 2. Sequential numbering of steps. 3. Clear actor-system interaction format.	1. Pre/post conditions are logical. Main flow achieves the use case goal. 2. Alternative flows handle exceptions appropriately. 3. Steps describe meaningful interactions.	P2	1. Each specification matches a use case in the model. 2. Actor roles match those in use case model.
	P4: Create a domain model based on use case specifications.	1. Classes shown with proper notation. 2. Relationships with correct multiplicity notation. 3. Proper relationship types. 4. Attributes listed within classes. 5. No design decisions (visibilities, navigabilities, data types)	1. Classes represent meaningful domain concepts. 2. Relationships reflect real-world connections. 3. Multiplicities reflect actual constraints.	P3	1. Classes represent nouns from use case specifications. 2. Relationships reflect interactions described in specifications. 3. Attributes capture data mentioned in specifications. 4. Multiplicities reflect constraints in specifications.
	P5: Identify system operations from use case specifications.	1. Operation signature format (name, parameters, return type). 2. Parameter types specified. 3. Return types specified.	1. Operations reflect system responsibilities. 2. Parameters capture necessary inputs. 3. Return types appropriate for operation purpose. 4. Operations support use case functionality.	P4	1. Operations cover all user-initiated functions in use cases. 2. Parameters match data exchanges in specifications. 3. Return types align with expected outcomes.

RA: Requirements Analysis, SD: System Description.

class diagrams in P4 in that they define the internal structure of the system with design decisions made, whereas domain class diagrams describe the context of the system in terms of real-world concepts without any design decisions.

The Implementation phase involves activity P8 which focuses on creating an implementation in Java, the chosen language for this study. Validity requires concrete code for classes including attributes, constructors, getters, setters, and method implementations, and consistency is checked against the attributes and operations in the design class diagram in P7. In the Testing phase, activity P9 aims to develop test cases with validity requiring concrete code for unit, integration, and system test methods with assertions, and consistency checked against the class implementation in P8.

The evaluation of model outputs was conducted independently by both authors, who bring extensive experience in software engineering research and development. Following the criteria defined in Table 1 and Table 2, each author assessed the artifacts' validity and consistency, with any assessment differences reconciled through joint review to ensure evaluation reliability. For similarity analysis, we utilized LLMs to assist in identifying common and unique properties between model

outputs, followed by manual verification of the identified properties by both authors. LLMs were chosen due to their superiority over traditional text-based similarity metrics (e.g., cosine similarity, Jaccard similarity) in capturing semantic and structural relationships [39,40]. The transcript of model outputs and the detailed reliability analysis are available in a public repository [41].

4.1. Reliability evaluation

In this work, we use the weights in Table 3 for the reliability metrics in Section 3.1. For RMA, Validity (V) and Consistency (C) are equally weighted at 0.5 each, representing their comparable importance towards reliability. In RMP, the Requirements Analysis (RA) phase assigns the same weight of 0.2 to each of P1–P5, indicating their equal significance. In the Design Modeling (DM) phase, weights are distributed as 0.6 for P6 and 0.4 for P7, reflecting the greater emphasis on design decisions in P6. The Implementation (I) and Testing (T) phases assign a full weight of 1.0 to their respective activities, P8 and P9. Lastly, the RMPR allocates weights of 0.4, 0.3, 0.2, and 0.1 to the RA, DM, I, and T phases, respectively, reflecting the increasing importance of the

Table 2
Development Activities: P6–P9.

Phase	Activities (Prompts)	Validity		Consistency	
		Structural rules	Intended meaning	Fit	Alignment criteria
DM	P6: Create design sequence diagrams for system operations.	1. Lifelines represent existing objects. 2. Proper message notation with arrows. 3. Return messages shown where appropriate.	1. Interactions achieve operation goals. 2. Message sequence is logical. 3. Object collaborations make sense.	P5	1. Messages implement identified system operations. 2. Parameters match operation signatures. 3. Return values align with operation specifications.
	P7: Create design class diagrams based on the domain model and sequence diagrams.	1. Classes with attributes and operations. 2. Proper relationship notation. 3. Visibility indicators. 4. Complete method signatures. Navigabilities, Data types.	1. Classes have clear responsibilities. 2. Relationships support required interactions. 3. Methods implement required operations. 4. Design principles (encapsulation, cohesion) followed	P4, P6	1. Classes maintain or refine domain model structure. 2. Operations match sequence diagram messages. 3. Relationships support message passing. 4. Method signatures match sequence. Interactions.
I	P8: Develop a Java Implementation for the system as specified in the class diagram and sequence diagrams.	1. Syntactically correct Java code. 2. Classes with proper structure (constructor, setters, getters). 3. Method signatures matching design.	1. Code implements design specifications. 2. Logical algorithms. 3. Appropriate error handling.	P6, P7	1. Classes implement design specifications. 2. Method signatures match design. 3. Relationships implemented as specified. 4. Visibility matches design modifiers. 5. Message sequences implemented as designed. 6. Method implementations follow interaction order in design. 7. Object collaborations follow sequence diagram lifelines.
T	P9: Develop tests including unit tests, integration tests, and system tests for the implementation.	1. Test method signature format. 2. Assertions properly formatted. 3. Test setup/teardown properly structured.	1. Tests verify required functionality. 2. Integration tests verify object interactions. 3. System tests validate use case scenarios.	P8	1. Unit tests verify implemented methods. 2. Integration tests check implemented relationships. 3. System tests validate use case scenarios. 4. Test data reflects valid input ranges.

DM: Design Modeling, I: Implementation, T: Testing.

Table 3
Weights distribution for reliability metrics across development activities.

Metric	RMA		RMP (RA)					RMP (DM)		RMP (I)	RMP (T)		RMPR			
	V	C	P1	P2	P3	P4	P5	P6	P7	P8	P9		RA	DM	I	T
Weight	0.5	0.5	0.2	0.2	0.2	0.2	0.2	0.6	0.4	1.0	1.0		0.4	0.3	0.2	0.1
Sum	1		1					1		1	1		1			

RA: Requirements Analysis, DM: Design Modeling, I: Implementation, T: Testing.

earlier development phases. These weights represent our decisions for this study and are not absolute – they can be adjusted based on different project contexts, organizational priorities, development methodologies, or specific quality requirements of individual software projects.

4.1.1. Reliability in FODS

Fig. 3 presents the reliability metrics for the FODS.

- **ChatGPT.** Table (a) presents the results for ChatGPT where on the row of RMA and RMP, the highest values are denoted in green and the lowest values are denoted in orange. The data shows that activities P2, P5, P6, and P9 achieved the highest RMA score of 1, whereas P8 registered the lowest score at 0.70. The high RMA score in P2 was due to the well-structured use case model, which included all necessary components (e.g., actors, use cases, relationships) and thoroughly covered the requirements identified in P1. Similarly, the high score in P5 was attributed to the precise definition of system operations with parameters and return specified, and consistency with the operations detailed in

the use case specifications in P3. The peak score in P6 resulted from incorporating essential components for sequence diagrams, such as participants and operations in sequence, and completely covering the system operations identified in P5. The high RMA score in P9 was due to the generation of concrete test code and the strong consistency of test cases with the class implementation in P8. The lowest score in P8 was attributed to incomplete method implementations and the absence of constructors, setters, and getters. For example, Fig. 4 shows the code snippet generated for the *DeliveryPerson* class. This code demonstrates several incompleteness issues. While the class properties are properly declared, it lacks actual implementations of constructors, getters, and setters (only indicated through a comment). Also, the *updateOrderStatus* method has only a placeholder comment instead of concrete implementation.

At the phase level, the Testing phase achieved the highest RMP score at 1, due to its single activity with the highest RMA score. Overall, ChatGPT achieved an RMPR score of 0.88 across an 82-page transcript.

A food order and delivery system should offer a convenient and efficient way for users to explore, order, and receive meals from a wide range of local and international cuisines. It should provide an accessible and easy-to-navigate interface, available on both web and mobile platforms, where users can browse menus, read reviews, and customize orders to their preferences. The system should feature real-time order tracking, estimated delivery times, and notifications to keep users updated from the moment an order is placed until it is delivered. Integration with various payment gateways should ensure secure and flexible payment options. Additionally, the system should support promotional features, loyalty programs, and personalized recommendations based on user preferences and order history, enhancing the overall user experience and encouraging repeat business.

(a) Food Order and Delivery System

A smart wallet is a next-generation digital wallet that combines traditional payment functionality with advanced blockchain technology and programmable features. Unlike basic cryptocurrency wallets that simply store and transfer digital assets, smart wallets can execute complex operations through smart contracts, automate transactions, and interact directly with decentralized applications. They offer enhanced security through features like multi-signature authentication, social recovery options, and customizable spending limits. Smart wallets also enable users to manage multiple cryptocurrencies, tokens, and NFTs in one interface, while providing advanced features like account abstraction, gas fee optimization, and batch transactions. This technology is revolutionizing how users interact with Web3 platforms by simplifying complex blockchain operations and offering a more user-friendly experience comparable to traditional financial services.

(b) Smart Wallet System

Fig. 2. Case studies.

- **Claude.** Table (b) presents the results for Claude, showing high RMA scores across activities. Activity P8 shows a relatively low RMA score at 0.84, which further analysis attributes to incomplete implementation for several classes and methods with the absence of constructors, getters, and setters. At the phase level, the Testing phase achieves the highest RMP score at 1, driven by its sole activity P9, which scored highest in RMA due to the development of concrete test cases that were highly consistent with the implementation in P8. Overall, Claude achieves an RMPR score of 0.95 across the longest transcript of 49 pages among the models.
- **Copilot.** Table (c) presents the results for Copilot. It shows that activity P5 achieved the highest RMA score at 1, due to the structural integrity of system operations identified and their consistency with the use case specifications in P4, although the operations are described more descriptively than definitively. On the other hand, activity P9 recorded the lowest RMA score at 0.40, primarily due to the lack of concrete test methods and inconsistencies in integration and system tests with the implementation in P8. A significant error was observed in the design class diagram in P7, concerning consistency with the domain model in P4, where Copilot misinterpreted the relationship description “Relationships: Connects users, restaurants, menu items, orders,

notifications, and delivery personnel.” for the *System* class in the domain model as an operation and erroneously introduced an operation named “Connects()” in the *System* class of the design class diagram as shown in Fig. 5. This error highlights the importance of a thorough review of outputs by human developers before practical implementation. At the phase level, the Requirements Analysis phase achieved the highest RMP score at 0.90. Overall, the model has an RMPR score of 0.78 with a 14-page transcript.

- **Gemini.** Table (d) presents the results for Gemini, demonstrating that activities P2 and P4 achieved the highest RMA score at 1. In P2, the use case model was comprehensive, including all essential components such as system boundary, actors, use cases, and relationships between actors and use cases with consistent use cases aligned with the requirements identified in P1. In P4, the domain model effectively incorporated key components like domain classes, attributes, class relationships, and multiplicities, maintaining consistency with the use case specifications from P3. On the other hand, P6 recorded a poor RMA score of 0.34, primarily due to misaligned visual representations in sequence diagrams. Fig. 6 shows an example of the *Place Order* sequence diagram. The diagrams used improperly aligned dashed lines to represent lifelines and messages, which led to confusion in understanding the depicted interactions.

Attribute	Requirements Analysis					Design Modeling		Imple.	Testing
Metric	P1	P2	P3	P4	P5	P6	P7	P8	P9
Validity	0.91	1.00	0.95	0.60	1.00	1.00	0.54	0.40	1.00
Consistency	0.73	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
RMA	0.82	1.00	0.98	0.80	1.00	1.00	0.77	0.70	1.00
RMP	0.92					0.91		0.70	1.00
RMPR						0.88			

(a) ChatGPT

Attribute	Requirements Analysis					Design Modeling		Imple.	Testing
Metric	P1	P2	P3	P4	P5	P6	P7	P8	P9
Validity	1.00	1.00	0.94	0.74	1.00	0.87	0.65	0.32	0.20
Consistency	0.61	0.75	1.00	1.00	1.00	1.00	0.89	0.89	0.60
RMA	0.81	0.88	0.97	0.87	1.00	0.93	0.77	0.60	0.40
RMP	0.90					0.87		0.60	0.40
RMPR						0.78			

(c) Copilot

Attribute	Requirements Analysis					Design Modeling		Imple.	Testing
Metric	P1	P2	P3	P4	P5	P6	P7	P8	P9
Validity	1.00	1.00	0.95	1.00	0.75	0.48	0.54	0.32	0.30
Consistency	0.63	1.00	1.00	1.00	0.70	0.20	0.85	0.67	0.86
RMA	0.81	1.00	0.98	1.00	0.73	0.34	0.69	0.49	0.58
RMP	0.90					0.48		0.49	0.58
RMPR						0.66			

(d) Gemini

Attribute	Requirements Analysis					Design Modeling		Imple.	Testing
Metric	P1	P2	P3	P4	P5	P6	P7	P8	P9
Validity	1.00	0.90	0.95	1.00	0.63	0.96	0.52	0.32	0.20
Consistency	0.86	0.85	1.00	0.86	0.48	0.83	0.76	1.00	1.00
RMA	0.93	0.87	0.98	0.93	0.55	0.89	0.64	0.66	0.60
RMP	0.85					0.79		0.66	0.60
RMPR						0.77			

(e) Meta

Attribute	Requirements Analysis					Design Modeling		Imple.	Testing
Metric	P1	P2	P3	P4	P5	P6	P7	P8	P9
Validity	1.00	1.00	1.00	1.00	1.00	0.96	0.94	0.75	1.00
Consistency	0.89	1.00	1.00	1.00	1.00	1.00	0.94	0.93	1.00
RMA	0.94	1.00	1.00	1.00	1.00	0.98	0.94	0.84	1.00
RMP	0.99					0.96		0.84	1.00
RMPR						0.95			

(b) Claude

Fig. 3. FODS: Reliability by models.

```

public class DeliveryPerson {
    private int deliveryPersonID;
    private String name;
    private String phone;

    // Constructors, getters, and setters

    public void updateOrderStatus(int orderID, String status) {
        // Implementation to update order status
    }
}

```

Fig. 4. The *DeliveryPerson* Class implementation generated by ChatGPT.

9. System: - Represents the overall food order and delivery system. - Relationships: Connects users, restaurants, menu items, orders, notifications, and delivery personnel.
(a) The <i>System</i> Class Description from the Domain Class Diagram in P4
<pre> class System { +Connects(users, restaurants, menuItems, orders, notifications, deliveryPersonnel) } </pre>
(b) The <i>System</i> Class Definition from the Design Class Diagram in P7

Fig. 5. The *System* Class generated by Copilot.

Similar issues occurred in the visual representation of the design class diagram in P7, which resulted in a lower RMA score of 0.69. Gemini was instructed to describe the visual representations in text, but it struggled to accurately interpret them. It was the only model producing visual designs. At the phase level, the Requirements Analysis phase achieved the highest RMP score of 0.90, while the Design Modeling phase recorded the lowest RMP score at 0.48, due to the misaligned diagrams in P6 and P7.

Overall, Gemini demonstrated an RMPR score of 0.66, the lowest among the models with a 16-page transcript.

- **Meta.** Table (e) presents the results for Meta. It shows that activity P3 achieved the highest RMA score at 0.98, largely due to the detailed use case specifications that included necessary items such as pre-conditions, post-conditions, main success scenario, and alternative scenarios, consistent with the use cases in P3. On the other hand, activity P5 recorded the lowest RMA score

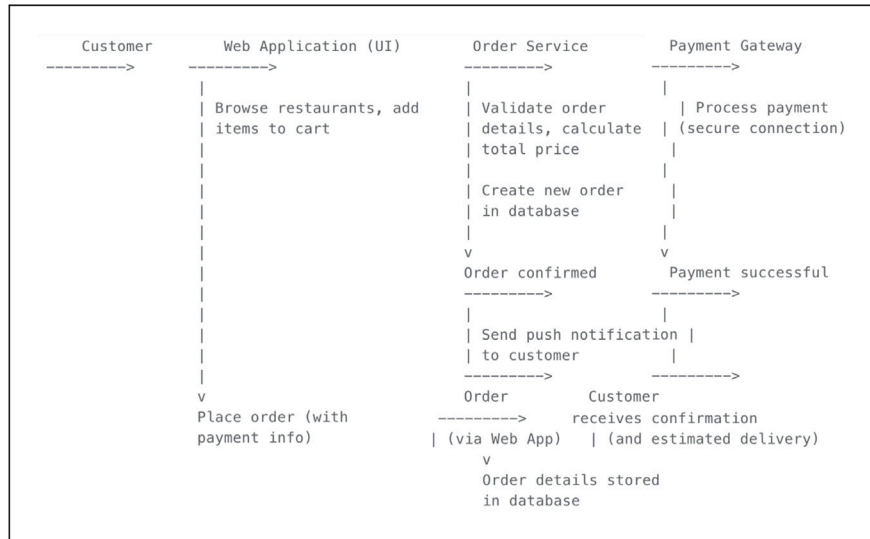


Fig. 6. Place Order Design sequence diagram generated by Gemini.

at 0.55, primarily due to significant inconsistencies in system operations with the use case specifications from P4, where 12 out of 23 (52%) system operations were not covered by the use case specifications. At the phase level, the Requirements Analysis phase achieved the highest RMP score at 0.85, while the Testing phase recorded the lowest RMP score at 0.60. The low RMP in the Testing phase was mainly attributed to the lack of implementation in test methods and unclear collaborations in integration and system tests. Overall, Meta demonstrated an RMPR score of 0.77, comparable to that of Copilot with a 5-page transcript, which is the shortest among the models.

Fig. 7 shows a comparison of the models in terms of RMA, RMP, and RMPR. In each column of Table (a), the highest values are highlighted in green and the lowest in orange. The data shows that Claude outperforms other models in almost all activities, except for P6, and leads in all phases with the highest RMPR of 0.95. In contrast, Gemini exhibited the weakest performance, recording the lowest RMA scores in several activities and achieving the lowest overall RMPR of 0.66. ChatGPT showed strong performance, ranking second and surpassing Claude in P6 with competitive results in P2, P5, and P9. Both Copilot and Meta displayed moderate performance with RMPR scores of 0.78 and 0.77, respectively.

In terms of average RMAs, activity P3 achieves the highest score at 0.98, demonstrating the models' proficiency in detailing use case specifications, while activity P9 records the lowest score at 0.72, indicating a greater need for human developer involvement. A similar observation is made with RMPs, where the Requirements Analysis phase achieves the highest score at 0.91, and the Testing phase the lowest at 0.72. Overall, the models achieve an average RMPR score of 0.81. These findings suggest that while some models show impressive performance, they are still prone to errors that require human oversight and review before their outputs are implemented.

4.1.2. Reliability in SWS

Fig. 8 presents the reliability metrics for the SWS.

- **ChatGPT.** Table (a) shows that activities P2 and P9 achieved the highest RMA score of 1.00. The high score in P2 is attributed to the well-structured use case model, which includes all necessary components and full coverage of the requirements identified in P1. Conversely, P5 registered a lower RMA score of 0.77 due to high invalidity in identified operations which include many internal operations which are not user-initiated in the use cases

in P3. For example, Fig. 9 shows the *Execute Smart Contracts* use case in P3 and the identified system operations for the use case in P5. From the user's perspective, the only valid system operation is the *executeSmartContracts*, which the user initiates. The other operations, *verifyContractExecution* and *recordContractTransaction*, are internal processes executed subsequently by the system, which should primarily be concerned during the design modeling phase. With respect to RMP, high performance is observed across phases. The slightly lower RMP score of 0.9 in the Requirements Analysis phase is due to low consistency in non-functional requirements in P1 with the system description and the low consistency of system operations in P5 with P4. The highest score of 1 in P9 is attributed to the comprehensive coverage of all three aspects of testing – unit, integration, and system tests – with concrete test implementations. Overall, ChatGPT achieved an RMPR score of 0.92, signifying high reliability across the system development, with a 59-page transcript.

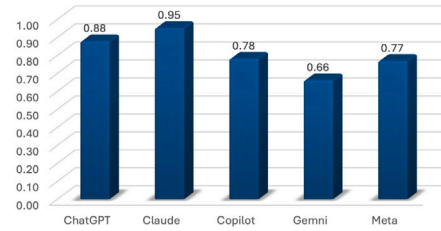
- **Claude.** Table (b) presents the results for Claude, demonstrating consistently decent RMA scores across activities. Activity P7 shows a relatively low RMA score at 0.81 due to missing core classes such as *Asset* and *SmartContract*, and inconsistencies where manager classes referenced as participants in the design sequence diagrams in P6 are not defined in the class diagram. On the other hand, P3 and P9 demonstrated the highest score of 1, with P3's score attributed to the high-quality use case specifications with rigorous structure and high consistency with P2, and P9's score due to the concrete implementation of test cases. At the phase level, the Implementation phase registered a relatively lower score at 0.82 due to missing setters and the inclusion of a class (*Balance*) that was not part of the design in P6 and P7, representing an inconsistency. Overall, Claude achieves an RMPR score of 0.88 across the longest transcript of 119 pages among the models.

Copilot. Table (c) presents the results for Copilot. It shows that activity P2 achieved the highest RMA score at 1, due to the structural integrity of the use case model and its high consistency with the functional requirements in P1. Conversely, activity P8 recorded the lowest RMA score at 0.56, primarily due to the incomplete implementation of methods in classes. Influenced by these RMA scores, the Requirements Analysis phase shows the highest RMP score at 1, while the Implementation phase registered the lowest RMP score, which matches the RMA score of P8. Overall, the model has an RMPR score of 0.81 with a 32-page transcript.

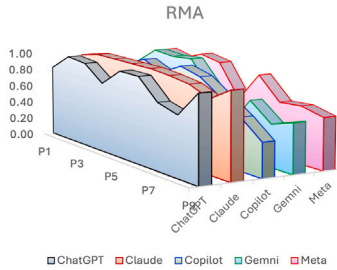
Model	RMA									RMP				RMPR
	P1	P2	P3	P4	P5	P6	P7	P8	P9	RA	DM	I	T	
ChatGPT	0.82	1.00	0.98	0.80	1.00	1.00	0.77	0.70	1.00	0.92	0.91	0.70	1.00	0.88
Claude	0.94	1.00	1.00	1.00	1.00	0.98	0.94	0.84	1.00	0.99	0.96	0.84	1.00	0.95
Copilot	0.81	0.88	0.97	0.87	1.00	0.93	0.77	0.60	0.40	0.90	0.87	0.60	0.40	0.78
Gemni	0.81	1.00	0.98	1.00	0.73	0.34	0.69	0.49	0.58	0.90	0.48	0.49	0.58	0.66
Meta	0.93	0.87	0.98	0.93	0.55	0.89	0.64	0.66	0.60	0.85	0.79	0.66	0.60	0.77
Average	0.86	0.95	0.98	0.92	0.86	0.83	0.76	0.66	0.72	0.91	0.80	0.66	0.72	0.81

(a) RMA scores for activities P1-P9, RMP scores for phases Requirements Analysis (RA), Design Modeling (DM), Implementation (I), and Testing (T) phases, and overall RMPR scores for each model. Highest values are highlighted in green and lowest in orange.

RMPR

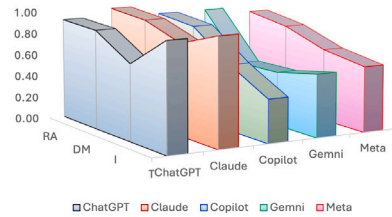


(b) Comparing overall RMPR scores across all five models



(c) RMA scores across all nine activities (P1-P9) for each model, illustrating performance variations throughout different development activities.

RMP



(d) Comparing RMP scores across the four development phases (RA, DM, I, T) for each model, demonstrating phase-level performance patterns.

Fig. 7. FODS: Reliability comparison.

Attribute	Requirements Analysis					Design Modeling		Imple.	Testing
Metric	P1	P2	P3	P4	P5	P6	P7	P8	P9
Validity	1.00	1.00	0.93	0.90	0.88	0.96	0.85	0.92	1.00
Consistency	0.67	1.00	1.00	1.00	0.67	0.83	1.00	1.00	1.00
RMA	0.83	1.00	0.97	0.95	0.77	0.89	0.92	0.96	1.00
RMP	0.90					0.91		0.96	1.00
RMPR	0.93								

(a) ChatGPT

Attribute	Requirements Analysis					Design Modeling		Imple.	Testing
Metric	P1	P2	P3	P4	P5	P6	P7	P8	P9
Validity	1.00	0.90	1.00	0.70	0.88	0.87	0.90	0.75	1.00
Consistency	0.65	0.88	1.00	1.00	0.79	1.00	0.73	0.89	1.00
RMA	0.83	0.89	1.00	0.85	0.83	0.93	0.81	0.82	1.00
RMP	0.88					0.89		0.82	1.00
RMPR	0.88								

(b) Claude

Attribute	Requirements Analysis					Design Modeling		Imple.	Testing
Metric	P1	P2	P3	P4	P5	P6	P7	P8	P9
Validity	1.00	1.00	0.94	0.74	1.00	0.87	0.67	0.92	0.67
Consistency	0.81	1.00	1.00	0.96	0.88	1.00	0.67	0.20	0.90
RMA	0.91	1.00	0.97	0.85	0.94	0.93	0.67	0.56	0.78
RMP	0.93					0.83		0.56	0.78
RMPR	0.81								

(c) Copilot

Attribute	Requirements Analysis					Design Modeling		Imple.	Testing
Metric	P1	P2	P3	P4	P5	P6	P7	P8	P9
Validity	1.00	0.00	0.95	1.00	0.63	0.74	0.62	0.70	0.77
Consistency	0.73	0.80	1.00	1.00	0.59	0.40	1.00	1.00	1.00
RMA	0.87	0.40	0.98	1.00	0.61	0.57	0.81	0.85	0.88
RMP	0.77					0.66		0.85	0.88
RMPR	0.77								

(d) Gemini

Attribute	Requirements Analysis					Design Modeling		Imple.	Testing
Metric	P1	P2	P3	P4	P5	P6	P7	P8	P9
Validity	1.00	0.80	0.80	0.80	0.63	0.91	0.52	0.36	0.73
Consistency	0.94	0.88	1.00	1.00	0.45	0.64	0.58	1.00	0.50
RMA	0.97	0.84	0.90	0.90	0.54	0.78	0.55	0.68	0.62
RMP	0.83					0.69		0.68	0.62
RMPR	0.74								

(e) Meta

Fig. 8. SWS: Reliability by models.

Gemini. Table (d) presents the results for Gemini, showing varying RMA scores across activities. P2 recorded the lowest score at 0.4, primarily due to the incorrect generation of a class diagram instead of a use case diagram. Fig. 10 shows a partial class diagram incorrectly generated by Gemini in response to P2's prompt "Create a use case diagram for the system", where it produced a class diagram instead of the requested use case

diagram. This error underscores the critical need for human review of LLM outputs with precise domain knowledge. Such an error in the development process could significantly impact the project negatively. Conversely, P4 achieved the highest score of 1, attributed to the proper notation in the domain model and its high consistency with the use case specifications in P3. At the phase level, the Design Modeling phase posted the lowest score at 0.66,

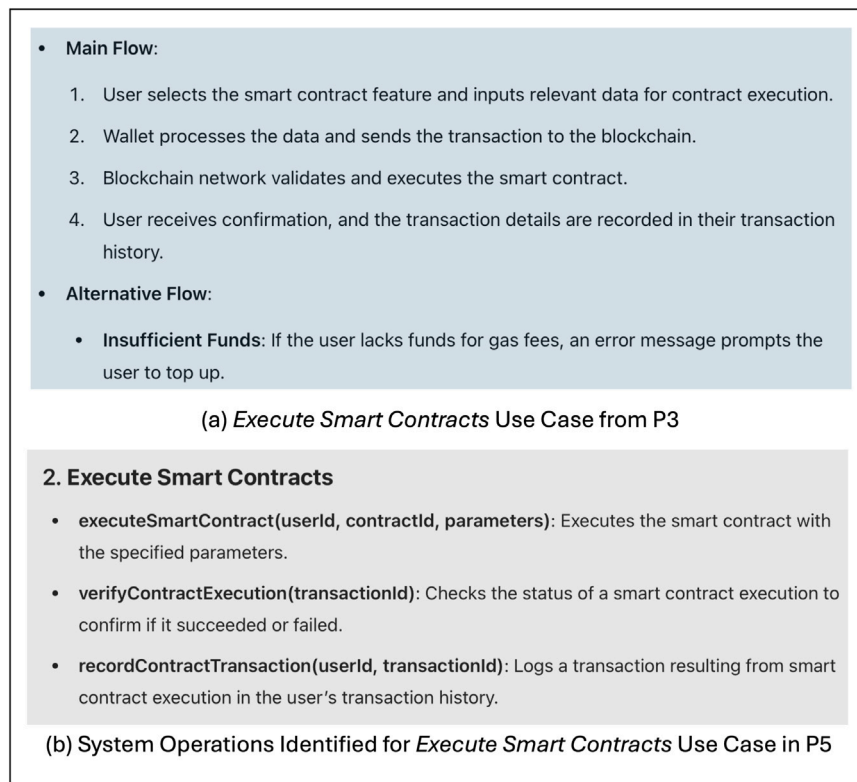


Fig. 9. *Execute Smart Contracts* Use case and identified system operations by ChatGPT.

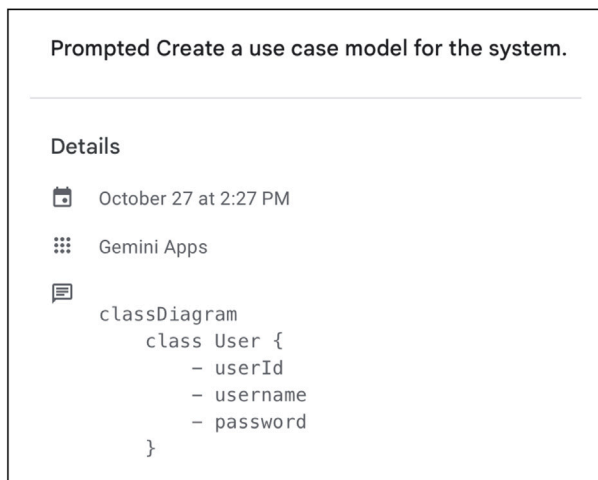


Fig. 10. A portion of the class diagram produced in reponse to P2 for creating use case diagram in Gemini.

mainly because it generated system sequence diagrams instead of design sequence diagrams. Although similar in notation, they differ in purpose and context, necessitating domain expertise to distinguish between them. P9 recorded the highest RMP score at 0.88, despite the lack of concrete test implementations for system tests (providing only textual descriptions). Overall, Gemini demonstrated an RMPR score of 0.77, which is relatively lower than other models, with a 24-page transcript.

- **Meta.** Table (e) presents the results for Meta, showing that activities in the Requirements Analysis phase have higher scores

than those in other phases. Specifically, P1 achieved the highest RMA score at 0.97, due to the high consistency of both functional and non-functional requirements with the system description. Conversely, activity P5 recorded the lowest RMA score at 0.54 because of loosely defined system operations without specified base use cases, lacking parameters and returns, and the extensive stretching of system operations, resulting in many operations not covered in the use case specifications from P4. Despite the notably lower score in P4, the Requirements Analysis phase managed to achieve the highest RMA score of 0.84. Meanwhile, the Testing phase presented the lowest RMA score of 0.62 primarily due to the absence of implementation for system tests and poorly formed integration tests that only cover basic interactions between data classes and their managers, while neglecting to evaluate core system functionality. Fig. 11 shows the integration test generated for the *UserManager* class, which only tests basic operations like creating a *User* object and accessing a getter method, failing to evaluate any meaningful integration for core functions. Overall, Meta demonstrated an RMPR score of 0.74, the lowest among the models, accompanied by a 23-page transcript, which is the shortest among the models.

Fig. 12 provides a comparative analysis of the models' performances. The data indicates that ChatGPT achieves the highest RMPR of 0.92, outperforming other models in four out of nine activities and leading in most phases, with the exception of the Requirements Analysis phase. Conversely, Meta demonstrates the weakest performance, recording the lowest RMA scores in four activities and achieving the lowest overall RMPR of 0.74. Claude shows strong performance, ranking second and surpassing ChatGPT in activities P3 and P6, and tying with the highest score of 1 in activity P9. The remaining three models – Copilot, Gemini, and Meta – display moderate performance with RMPR scores of 0.81, 0.77, and 0.74, respectively. These results

And here is an example of what an integration test for the `UserManager` class might look like in Java:

```

Java

public class UserManagerTest {
    @Test
    public void testCreateUser() {
        UserManager userManager = new UserManager();
        User user = userManager.createUser("username", "password", "biometric");
        assertNotNull(user);
        assertEquals(1, user.getUserId());
    }
}

```

Fig. 11. An integration test generated by meta.

align with those from the FODS case, where ChatGPT and Claude notably outperformed Copilot, Gemini, and Meta.

Average RMA scores reveal that the models excel in developing use case specifications in activity P3, achieving the highest score of 0.96, whereas they are least proficient in identifying system operations in activity P5 with the lowest score of 0.74. This indicates a need for improvement in identifying system operations from use case specifications. More specifically, in P5, the models need to enhance their ability to systematically identify system operations through interactions between the actor and the system derived from the main success scenarios and alternative scenarios in use case specifications. Average RMPs show that the models are more proficient in requirements analysis and testing, each scoring the highest at 0.86, but less proficient in implementation with a score of 0.77. This performance is consistent with the FODS case study. Overall, the models achieve an average RMPR score of 0.82, which closely mirrors the RMPR score of 0.81 from the FODS study. These results reaffirm the necessity for human oversight and review of model outputs, as previously observed in the FODS case.

4.2. Similarity evaluation

In this section, we present the similarity analysis on the outputs of the five models across the activities described in Table 1 and Table 2 through the two case studies of FODS and SWS. Similarity is quantified at three levels – SMA, SMP, and SMPR, following the metrics as outlined in Section 3.2. For each system, we compare outputs between every pair of models (10 pairs total across the five models), scoring similarity on a 0-5 scale where 0 indicates completely different outputs and 5 represents nearly identical results. This comparative analysis reveals not only how models converge or diverge in their approaches but also how their similarity patterns shift between traditional and emerging technology domains. While detailed activity-level measurements and calculations are available in the replication package [41], here we focus on key patterns and insights organized by development phase.

4.2.1. Similarity in FODS

Fig. 13 presents SMA scores of activities, SMP scores of phases, the SMPR of the entire process, and the average similarity by models. For consistent comparison across models, we focused analysis on common elements – the “Place Order” use case and scenario in P3, P5, P6, and P9, and the “Order” class in P8.

The Requirements Analysis phase involving activities P1-P5 shows a clear pattern of decreasing similarity as activities progress from initial requirements to system operations. The highest similarity appears in requirements identification (P1: 3.6), where Meta demonstrates the strongest alignment with other models (4.0), particularly in functional requirements. This similarity gradually declines through use case modeling and specification, reaching its lowest point in system operations

identification (P5: 2.4). The substantial variation in operation counts (ChatGPT: 24, Claude: 46, Copilot: 8, Gemini: 27, Meta: 23) reveals fundamentally different approaches to system decomposition, despite starting from the same requirements base.

In the Design Modeling phase involving P6 and P7, models demonstrate moderate convergence in sequence diagrams (P6: 2.9) but show increasing divergence in class design decisions. Notably, design class diagrams (P7: 2.6) exhibit lower similarity than their domain model counterparts (P4: 3.1), suggesting that models incorporate more unique design decisions as they move from conceptual to technical design. This phase reveals how models transition from shared domain understanding to distinct architectural approaches.

The Implementation phase presents an interesting contrast in the *Order* class implementation, showing unexpectedly high similarity in class attributes (P8: 4.0) but significant variation in operation implementation. While models largely agree on core attributes, they differ substantially in behavioral aspects, with some models providing more operations (ChatGPT: 3 operations, Claude: 4 operations) and others focusing solely on structural elements (Copilot/Gemini: 0 operations). This suggests a shared understanding of data structure but divergent approaches to system behavior.

The Testing phase (P9: 2.7) reveals distinct approaches to quality assurance, with each model adopting unique strategies for test scenario design. This moderate similarity suggests that while models share some basic testing principles, they exhibit individual preferences in test coverage and methodology, reflecting their unique interpretations of system criticality and test priorities.

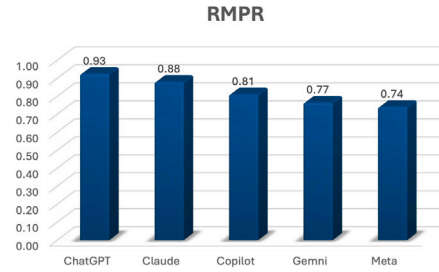
The analysis across development phases reveals interesting patterns in model similarity. Requirements Analysis exhibits the highest average similarity (2.96), highlighting models’ strong convergence in early phases. This is followed by Design Modeling (2.75), while Implementation shows interesting variation in individual outcomes despite a moderate similarity score of 2.70. The Testing phase demonstrates the lowest similarity (2.7), reflecting distinct quality assurance approaches. Overall, models demonstrate moderate similarity scores ranging from 2.97 to 3.11, with Meta showing slightly higher alignment (3.11) across phases and Copilot exhibiting more unique characteristics (2.97). The aggregate SMPR score of 3.04 translates to 61% similarity across all phases, indicating a significant 39% divergence in how models approach software development tasks. This balance between commonality and uniqueness suggests that while models share fundamental software engineering principles, they exhibit distinct characteristics that practitioners should consider when selecting models for specific development tasks.

4.2.2. Similarity in SWS

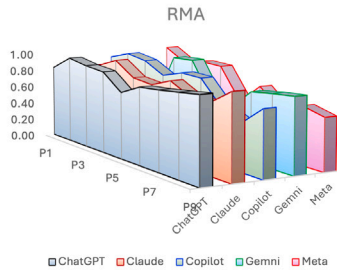
Fig. 14 presents SMA, SMP, and SMPR scores along with the average similarity by models. For consistent comparison across models, we

Model	RMA									RMP				RMPR
	P1	P2	P3	P4	P5	P6	P7	P8	P9	RA	DM	I	T	
ChatGPT	0.83	1.00	0.97	0.95	0.77	0.89	0.92	0.96	1.00	0.90	0.91	0.96	1.00	0.93
Claude	0.83	0.89	1.00	0.85	0.83	0.93	0.81	0.82	1.00	0.88	0.89	0.82	1.00	0.88
Copilot	0.91	1.00	0.97	0.85	0.94	0.93	0.67	0.56	0.78	0.93	0.83	0.56	0.78	0.81
Gemni	0.87	0.40	0.98	1.00	0.61	0.57	0.81	0.85	0.88	0.77	0.66	0.85	0.88	0.77
Meta	0.97	0.84	0.90	0.90	0.54	0.78	0.55	0.68	0.62	0.83	0.69	0.68	0.62	0.74
Average	0.88	0.83	0.96	0.91	0.74	0.82	0.75	0.77	0.86	0.86	0.79	0.77	0.86	0.82

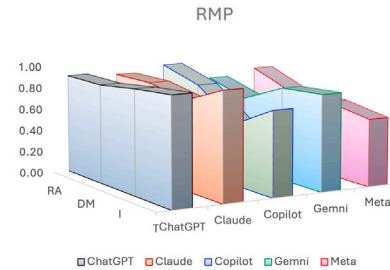
(a) RMA scores for activities P1-P9, RMP scores for phases Requirements Analysis (RA), Design Modeling (DM), Implementation (I), and Testing (T) phases, and overall RMPR scores for each model. Highest values are highlighted in green and lowest in orange.



(b) Comparing overall RMPR scores across all five models



(c) RMA scores across all nine activities (P1-P9) for each model, illustrating performance variations throughout different development activities.



(d) Comparing RMP scores across the four development phases (RA, DM, I, T) for each model, demonstrating phase-level performance patterns.

Fig. 12. SWS: Reliability comparison.

focused analysis on common elements – the “Transfer Assets” use case and scenario in P3, P6, and P9; the “Manage Assets” use case in P5; and the “Transaction” class in P8.

The Requirements Analysis phase (P1-P5) demonstrates a notable decline in similarity as models progress deeper into blockchain-specific requirements. Initial requirements gathering shows high similarity (P1: 4.0), with an interesting contrast between Copilot’s strict adherence to specifications (4.5) and Claude’s domain-enriched approach (3.25). This similarity drops sharply through use case modeling (P2: 3.5) to system operations (P5: 1.8), revealing increasingly divergent interpretations of digital asset management requirements and features.

During the Design Modeling phase (P6-P7), significant variations emerge in technical depth and modeling approaches. Sequence diagrams (P6: 3.0) reveal a clear divide between models emphasizing detailed smart wallet components (e.g., ChatGPT/Claude including WalletService, BlockchainNetwork) and those maintaining simpler interactions (Copilot, Gemini, Meta). Design class diagrams show particularly low similarity (P7: 2.2), reflecting fundamentally different approaches to system organization.

The Implementation phase exhibits notably low similarity (P8: 2.0) in the *Transaction* class implementation, with models diverging significantly in both property selection and operation definitions. This phase reveals fundamental differences in how models conceptualize smart wallet functionality, from basic transaction handling to advanced smart contract interactions, suggesting varying levels of domain expertise.

The Testing phase shows the lowest similarity (P9: 2.2) across models, exemplified by dramatically different approaches to transaction testing sequences. For example, Claude’s testing covers the comprehensive transaction lifecycle including blockchain-specific steps (transfer initiation, transaction creation, validation, balance verification, gas fee estimation, confirmation, execution, network broadcasting), versus

Meta’s testing focuses only on basic transaction management (transaction request, basic validation, balance verification, creation, confirmation, balance update, and recording) while omitting critical blockchain operations like gas fee handling and network broadcasting. These variations reveal notable differences in how models understand and verify smart wallet operations, reflecting their distinct interpretations of smart wallet system validation requirements.

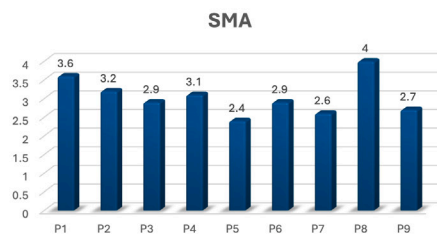
Looking across models in the smart wallet domain reveals generally modest similarity, with scores ranging from 2.47 to 2.83. Within this range, Copilot demonstrates the highest similarity (2.83), maintaining more consistent approaches across development phases, while Gemini shows the lowest (2.47), exhibiting more unique interpretations of the development. Phase-level analysis reveals a clear pattern. The Requirements Analysis phase achieves the highest similarity (2.96), anchored by strong initial agreement in P1 (4.0) where models work from the same system description. However, this similarity decreases sharply through subsequent activities, particularly in system operations identification (P5: 1.8), suggesting divergent interpretations of smart wallet functionality. The Design Modeling and Implementation phases show progressively lower similarity scores, with Implementation scoring lowest (2.0), as models demonstrate distinctly different approaches to the implementation of smart wallet functionality. Overall, the models achieve an SMPR score of 2.69, translating to 54% similarity with 46% uniqueness, indicating notable variation in how models approach smart wallet development. This degree of divergence particularly emerges in technical implementation details and testing strategies, suggesting that distinct capabilities and domain expertise among different models in emerging technology domains.

4.2.3. Analysis

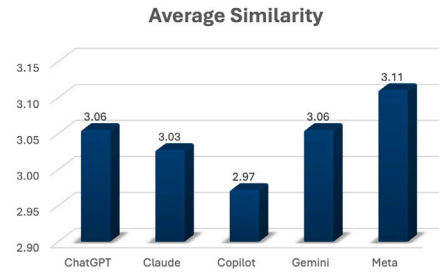
In this section, we present a combined analysis that consolidates the results of the FODS and SWS case studies, examining patterns and insights across both studies in terms of reliability and similarity.

Models	Requirements Analysis					Design Modeling		Imple.	Testing	
Metric	P1	P2	P3	P4	P5	P6	P7	P8	P9	Average
ChatGPT	3.75	3.00	3.00	2.75	2.75	3.00	2.50	3.75	3.00	3.06
Claude	3.50	3.25	3.25	3.25	1.75	3.00	2.50	4.25	2.50	3.03
Copilot	3.50	3.00	3.00	3.00	1.75	3.25	2.50	4.00	2.75	2.97
Gemini	3.25	3.25	3.00	3.00	3.25	2.75	2.50	3.75	2.75	3.06
Meta	4.00	3.50	2.25	3.50	2.50	2.50	3.00	4.25	2.50	3.11
SMA	3.60	3.20	2.90	3.10	2.40	2.90	2.60	4.00	2.70	
SMP							2.75	4.00	2.70	
SMRP						3.04				

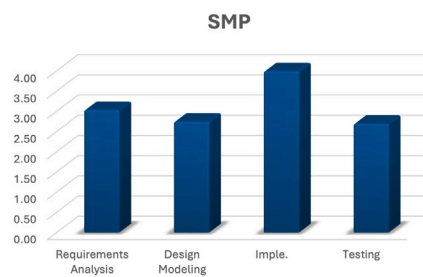
(a) SMA scores for activities P1-P9 across each model, SMP scores for Requirements Analysis, Design Modeling, Implementation, and Testing phases, and overall SMRP score. Notable scores are highlighted, with highest values in green and lowest in yellow.



(c) SMA scores across all nine activities (P1-P9).



(b) Comparing average similarity scores across all five models.

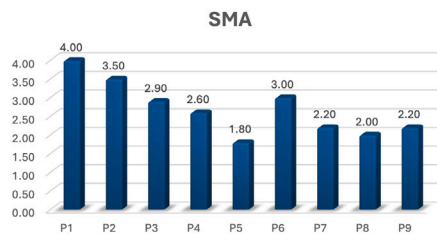


(d) SMP scores across the four development phases.

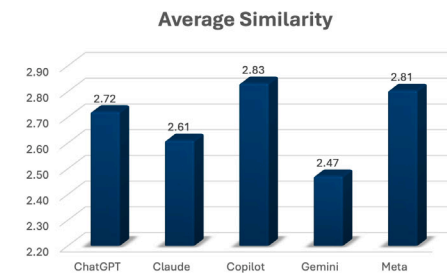
Fig. 13. FODS: Similarity analysis.

Models	Requirements Analysis					Design Modeling		Imple.	Testing	
Metric	P1	P2	P3	P4	P5	P6	P7	P8	P9	Average
ChatGPT	4.00	3.75	2.50	3.00	1.50	2.75	2.75	2.00	2.25	2.72
Claude	3.25	3.50	2.75	2.50	2.25	2.75	2.25	2.00	2.25	2.61
Copilot	4.50	4.00	3.50	2.75	1.75	3.25	2.00	1.75	2.00	2.83
Gemini	4.00	3.00	2.75	2.00	1.75	3.00	1.50	2.00	2.25	2.47
Meta	4.25	3.25	3.00	2.75	1.75	3.25	2.50	2.25	2.25	2.81
SMA	4.00	3.50	2.90	2.60	1.80	3.00	2.20	2.00	2.20	
SMP							2.60	2.00	2.20	
SMRP						2.69				

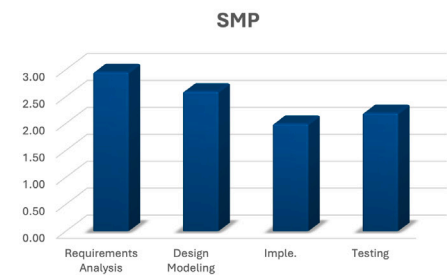
(a) SMA scores for activities P1-P9 across each model, SMP scores for Requirements Analysis, Design Modeling, Implementation, and Testing phases, and overall SMRP score. Notable scores are highlighted, with highest values in green and lowest in yellow.



(c) SMA scores across all nine activities (P1-P9).

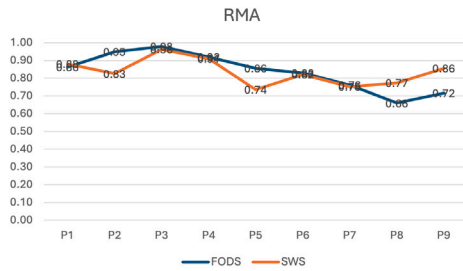


(b) Comparing average similarity scores across all five models.

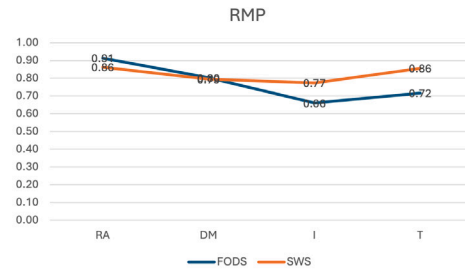


(d) SMP scores across the four development phases.

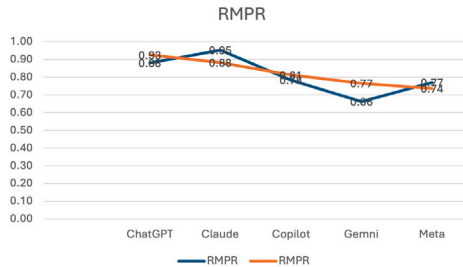
Fig. 14. SWS: Similarity analysis.



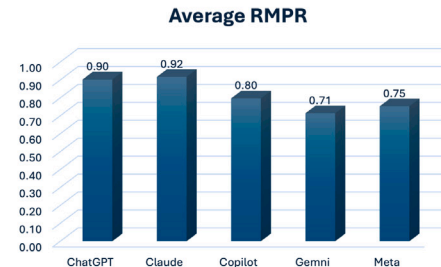
(a) Comparing RMA scores between FODS and SWS across all nine activities (P1-P9).



(b) Comparing RMP scores between FODS and SWS across the four development phases (RA, DM, I, T).



(c) Comparing RMPR scores between FODS and SWS for each model, demonstrating how individual models perform across both case studies.



(d) Average RMPR scores across both case studies for each model.

Fig. 15. Reliability analysis.

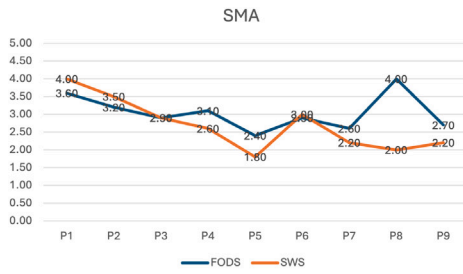
Reliability analysis. The two case studies on fundamentally different systems – FODS and SWS – yield very similar RMPR scores with FODS at 0.81 and SWS at 0.82. Fig. 15 provides a detailed analysis of reliability by comparing the two studies. Graph (a) illustrates the comparison of RMA scores across different project phases, showing a consistent trend across both studies. Both FODS and SWS exhibit higher reliability in earlier activities with a noticeable decline until P7-P8, followed by a notable increase in P9. This pattern suggests that models initially exhibit high reliability, possibly due to working from the same system description, but reliability decreases as the complexity of tasks increases, indicating challenges in applying appropriate strategies for developing subsequent artifacts.

The increase in reliability in the later activities could suggest that models are better trained in implementation and testing activities. Notably, a divergence occurs in P8, where FODS shows increasing reliability in implementation, contrasting with a decreasing trend in SWS. This difference highlights varying performance patterns between traditional business systems and emerging technology domains, though the specific reasons for this variation require further investigation. Graph (b) compares the RMP scores, reflecting trends similar to those observed in Graph (a). Viewing the data at the phase level smooths out the higher fluctuations in reliability scores of SWS seen at the activity level in Graph (a), resulting in SWS appearing to have less variation compared to FODS at the phase level. Graph (c) illustrates the RMPR trends for each model across the two case studies, highlighting the superior performance of ChatGPT and Claude compared to Copilot, Gemini, and Meta. This trend is further emphasized in Graph (d), which presents the average RMPR scores, affirming the strong results for ChatGPT and Claude. Based on these graphs, Claude and ChatGPT outperform the other three models by an average of 22.4% and 19.8%, respectively.

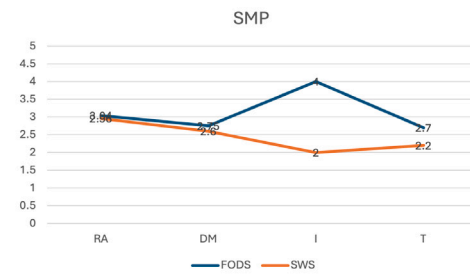
Similarity analysis. The two case studies result in similar SMPR scores with FODS at 3.04 and SWS at 2.69. Fig. 16 presents a detailed analysis comparing the similarity of the two case studies. Graph (a) displays the SMA scores, showing a generally similar trend except in P8. Both studies exhibit higher similarity in the initial activities, gradually decreasing until P5, which marks the end of the Requirements Analysis phase. This pattern of initial higher similarity and gradual decrease in the Requirements Analysis phase might be attributed to the models starting with the same system description, which they subsequently adapt as they incorporate their domain knowledge in developing subsequent artifacts.

The trends show increased similarity in P6 and P7 within the Design Modeling phase. This convergence might be attributed to models applying common software design principles, though further investigation is needed to understand the specific factors contributing to similar design decisions. A significant divergence occurs in P8 during the Implementation phase, where FODS demonstrates very high similarity, in contrast to SWS which shows markedly low similarity. The marked difference observed in P8 suggests potentially contrasting behaviors between traditional business systems and emerging technology domains, though further investigation would be needed to confirm this relationship. Both studies converge again in similarity by P9 in the Testing phase.

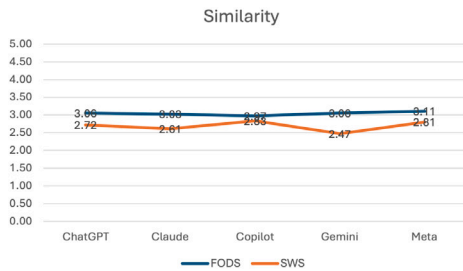
A similar trend to that seen in Graph (a) is observed in Graph (b), which presents similarity at the phase level. Graph (c), illustrating the similarity scores of models across two case studies, displays consistently lower similarity in SWS compared to FODS. This pattern of lower similarity in SWS might indicate that models take more diverse approaches when handling emerging technology domains compared to traditional business systems, though understanding the specific reasons for this difference requires further study. The graph reveals that while similarity across models in FODS is very uniform, there is a noticeable difference in SWS where Claude and Gemini exhibit slightly less similarity. This indicates that these models incorporated more unique artifacts in their



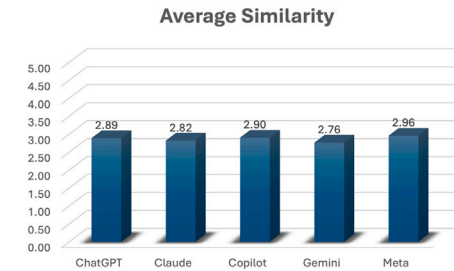
(a) Comparing SMA scores between FODS and SWS across all nine activities (P1-P9).



(b) Comparing SMP scores between FODS and SWS across the four development phases (RA, DM, I, T).



(c) Comparing overall similarity scores between FODS and SWS for each model. Shows consistently higher similarity scores in FODS compared to SWS.



(d) Average similarity scores across both case studies for each model.

Fig. 16. Similarity analysis.

outputs. Graph (d) presents the average similarity scores by model across the two case studies, indicating low similarity with a very narrow range from 2.76 to 2.96. This results in an overall average similarity of 2.87, translating to 57% similarity, with the 43% difference suggesting that each model retains distinct characteristics.

During the analysis, an interesting observation was made on the correlation between the size of model transcripts and reliability. Utilizing Pearson's correlation coefficient ($r = 0.949$), Fig. 17 visualizes the relationship between transcript size (in pages) and RMPR scores across models. It reveals a strong positive correlation as shown by the regression line. Claude leads in both metrics (84 pages, 0.92 RMPR), closely followed by ChatGPT (70.5 pages, 0.91 RMPR), while the other three models cluster at the lower end of both scales. The nearly linear arrangement of the points along the regression line suggests that models producing longer transcripts tend to achieve higher RMPR scores. This relationship accounts for approximately 90% of the variance in performance, as indicated by an r^2 value of 0.900. The pattern might suggest that more comprehensive outputs tend to be more reliable, likely because longer transcripts indicate more detailed explanations and comprehensive coverage of system aspects. However, it should be noted that transcript length alone does not guarantee quality – for example, Meta produced the shortest transcript (5 pages) but achieved a moderate reliability score (0.77), suggesting that conciseness can still be effective if the essential elements are properly addressed. The correlation is based on only five data points, and more data would be needed to establish a more definitive relationship between transcript size and reliability.

5. Discussion

In this section, we address the research questions posed in Section 1 and provide qualitative insights based on the findings in prior sections.

5.1. Response to research questions

RQ1: How can the reliability and similarity of software artifacts produced by LLMs be measured? We introduced a structured approach to measure the reliability and similarity of LLMs at the activity, phase, and overall process levels in software development. Reliability is assessed in terms of RMA at the activity level, RMP at the phase level, and RMPR at the process level (cf. Section 3.1). RMA evaluates the validity and consistency of produced artifacts; RMP is measured by weighted aggregation of RMAs; RMPR aggregates weighted RMPs (cf. Section 3.2). Similarity is measured in terms of SMA at the activity level, SMP at the phase level, and SMPR at the process level. SMA assesses the artifacts produced by models for individual activities, SMP averages the SMAs of activities within a phase, and SMPR involves a weighted average of the SMPs across the process. This structured approach allows for a systematic evaluation of models from fine granularity to the entirety of the process.

RQ2: How reliable are the software artifacts produced by LLMs? The experiment reveals an overall average RMPR of 0.82 for models across the two case studies. Claude led the performance, achieving an RMPR score of 0.92, followed by ChatGPT at 0.90, Copilot at 0.80, Meta at 0.75, and Gemini at 0.71 (cf. Fig. 15(d)). The results categorize the models into two distinct groups: a higher-performing group featuring Claude and ChatGPT, and a lower-performing group consisting of Copilot, Meta, and Gemini. Although the specific factors contributing to their performance levels are not immediately clear, a significant correlation was observed between reliability and the transcript size of the models (cf. Fig. 17). Notably, the transcripts of ChatGPT and Gemini are substantially longer than those of the other models. These findings indicate that while some models show reasonable reliability in their produced software artifacts, most still necessitate extensive human review before practical implementation.

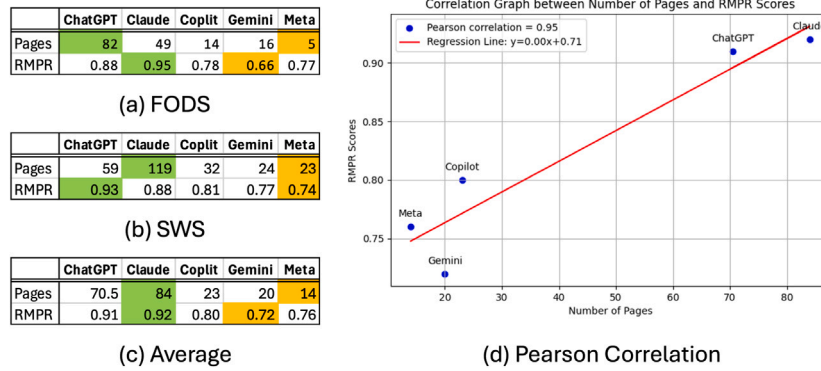


Fig. 17. Correlation between pages and RMPR.

RQ3: How similar are the software artifacts produced by LLMs to each other? The experiment reveals that the models exhibit overall low similarity with an average SMPR score of 2.87, ranging from 2.76 to 2.96, showing very slight variation across two case studies, which corresponds to a 57% similarity level (cf. Fig. 16(d)). This indicates that no model demonstrates exceptionally high similarity or variability compared to the others, yet there is a significant 43% uniqueness among them. Meta displays the closest similarity to other models with a score of 2.96, while Copilot is the most distinct with a score of 2.90, although their scores are relatively close. In terms of SMA, P1 recorded the highest score at 3.8, which focused on identifying requirements. This might be attributed to the fact that all models began with the same system description in the first activity. Subsequently, similarity gradually decreased in the subsequent activities within the Requirements Analysis phase as models incorporated more details from their own domain knowledge, making their outputs more unique. A notable difference was observed in P8, focusing on implementation in the two case studies where models showed high similarity in FODS, while significantly lower similarity in SWS. This contrast might indicate that models converge more in their implementation approaches for traditional business processes in FODS, while taking more diverse approaches when implementing emerging technologies in SWS, which involves emerging concepts such as blockchain, cryptocurrency, and smart contracts. The reasons for these different patterns in implementation approaches deserve further investigation. This variation in handling different domains, combined with the overall similarity analysis showing 43% uniqueness across model outputs, demonstrates that models possess distinct characteristics in producing software artifacts.

5.2. Qualitative insights

The quantitative results revealed several important qualitative insights about LLM behavior in software development tasks that have direct practical implications for both practitioners and researchers.

Notable differences are observed in model behaviors between the two systems, which might be attributed to the distinct characteristics of traditional business systems (FODS) versus emerging technology domains (SWS). This difference is most pronounced in implementation activities, where FODS shows increasing reliability (P8) while SWS exhibits decreasing reliability. The contrast extends to output similarity, with greater variations in SWS (54% similarity) compared to FODS (61% similarity), particularly in implementation and testing phases. These patterns suggest that while using LLMs in traditional business systems might lead to higher reliability and more consistent outputs, emerging technologies might require both more thorough review processes and different strategies to manage greater output variability.

Activity-specific behaviors reveal clear patterns that can guide resource allocation and task planning. Models excel at use case specifications (P3 average RMA: 0.98) and requirements analysis, suggesting

strong capabilities in understanding and structuring system behaviors. However, they consistently struggle with system operations identification (P5 average RMA: 0.74), indicating difficulties in systematically deriving operations from use cases. These varying capabilities are evident in specific errors, such as misinterpreting relationship descriptions in class diagrams (e.g., Copilot's error in FODS) or generating incorrect diagram types (e.g., Gemini's class diagram instead of use case diagram in SWS). Teams can use this knowledge to wisely allocate more review resources to areas known to be challenging, while sensibly using LLMs for tasks where they have shown consistent reliability.

Development phase characteristics provide a roadmap for effective LLM integration throughout the software lifecycle. Models consistently perform better in early development phases, particularly requirements analysis (RMP: 0.91 in FODS, 0.86 in SWS), suggesting strong capabilities in understanding and structuring initial system requirements. However, performance generally decreases as task complexity increases through design and implementation phases. This phase-dependent behavior suggests that teams might consider a graduated oversight approach, starting with lighter review processes in the early phases and gradually increasing human involvement as development progresses.

Model output patterns reveal important correlations that can inform tool selection and usage strategies. A strong correlation ($r = 0.949$) exists between transcript length and reliability, suggesting that models producing more detailed outputs (like ChatGPT and Claude) tend to achieve higher reliability scores. This variation in output comprehensiveness is reflected in the 43% variability across models, with some models like Claude providing extensive detail while others like Copilot offer more concise solutions. Teams can leverage these differences by selecting models based on task requirements – using comprehensive models for complex analysis tasks and more focused models for specific implementation tasks.

These findings have practical implications that go beyond choosing models to help guide broader development strategies. The varying reliability across activities and domains emphasizes the need for calibrated human oversight, particularly in complex design decisions and emerging technology domains. For optimal results, teams should adopt a hybrid approach: leveraging LLMs heavily in early development phases where they show consistent reliability, implementing structured review processes for known challenging activities, and maintaining strong human oversight in implementation and testing phases. The 43% variability in model outputs also suggests potential benefits in using multiple models for complex tasks to leverage their complementary strengths.

These findings reveal several promising areas for future research. First, investigating how model architectures and training approaches contribute to performance variations could reveal important insights about model capabilities and specialization. Second, deeper analysis of task-specific performance patterns could help understand why certain software development activities consistently show higher reliability

across models while others demonstrate lower reliability. Third, examining how models handle different application domains could clarify the factors influencing their adaptability to traditional versus emerging technologies. Fourth, systematic analysis of error patterns could reveal whether architectural and training choices lead to specific types of mistakes in software development tasks. Finally, investigating the impact of prompt engineering on model performance could provide crucial insights into optimizing LLM outputs. This includes studying how variations in prompt structure, specificity, and contextual information affect output reliability across different development phases and domains. These research directions could lead to more refined guidelines for LLM integration in software development processes and advance the understanding of how to optimize both model selection and utilization for specific development contexts.

6. Threats to validity

In this section, we discuss threats to validity in terms of internal validity, external validity, and construct validity.

Internal validity. Internal validity in this study faces several key threats. First, the subjective nature of assessing software artifacts, particularly in determining validity and consistency scores, could introduce researcher bias. Second, the study captures only the “first-attempt” performance of the models rather than their full potential when used iteratively to achieve more complete or refined outputs. For instance, if an initial output was incomplete or unsatisfactory, we did not attempt to improve it through follow-up prompts or clarifications, which would be typical in real-world software development scenarios. Thirds, this study uses a specific set of prompts for each activity, and the performance of LLMs could vary significantly with different prompt formulations. The way requirements are communicated, how design tasks are specified, or how implementation requests are phrased could all affect output quality. Even subtle variations in prompt structure, terminology, or level of detail might elicit different responses from the models, potentially leading to different reliability and similarity scores. Additionally, the use of non-paid versions of the models might not represent their full capabilities. To mitigate these threats, we established clear evaluation criteria and metrics before conducting the analysis, used a structured approach with well-defined formulas for calculations, maintained consistent evaluation standards across all models, and documented all evaluations in detail for transparency.

External validity. The main threats to external validity stem from the limited scope of the case studies and the LLMs selected, which may not represent all possible types of software development tasks or other LLMs available. To enhance generalizability, we applied the evaluation to two different systems – a food order and delivery system and a smart wallet system – covering both traditional and emerging domains. The models were assessed across various activities and phases of software development, from requirements analysis to testing, providing a comprehensive view of their capabilities and limitations in diverse contexts. However, the implementation analysis was limited to Java, and results may vary with other programming languages.

Construct validity. The primary threat to construct validity in this analysis is the potential mismatch between the metrics used (reliability and similarity) and the actual reliability and similarity of LLM outputs in software development. To mitigate this threat, we based the metrics on established object-oriented practices and software development processes, incorporated multiple levels of measurement (activity, phase, and process), and validated the metrics through two different case studies. We also provided clear definitions and calculations for each metric to ensure transparency and reproducibility of the measurement approach. However, we acknowledge that the metrics have several limitations: (1) the weight distributions in both reliability and similarity calculations lack empirical justification; (2) the 0-5 scale thresholds for

similarity mapping require empirical validation; and (3) the metrics do not include confidence measures to indicate the statistical significance of the results. Future work should address these limitations by incorporating validated weighting schemes, semantic analysis, and statistical confidence measures.

7. Conclusion

We evaluated the reliability and similarity of outputs from five popular LLMs – ChatGPT, Claude, Copilot, Gemini, and Meta – across different phases of software development. A structured approach is introduced to measure the reliability and similarity of software artifacts produced by these LLMs, applied through two case studies – a food order and delivery system and a smart wallet system. The results indicate that Claude performed the best in terms of reliability with ChatGPT also showing strong performance, while Gemini lagged behind and Copilot and Meta exhibited moderate performance. This suggests that while some LLMs demonstrate impressive performance, the majority still require thorough review by human developers before their outputs can be adopted in practice. In terms of similarity, the models demonstrated 57% similarity overall with 43% variability, presenting that each model has its unique characteristics that should be considered when adopting them. Further research should investigate factors influencing model performance variations in different domains, understanding task-specific reliability patterns, and examining how architectural choices and training approaches affect model capabilities in software development tasks. Future work should also extend the evaluation to industrial-level case studies using widely recognized benchmarks and frameworks that better represent real-world software development complexity. This could include evaluating LLMs on established open-source projects, industry-standard test suites, or standardized software engineering challenges that would provide a more comprehensive assessment of their capabilities in production environments. Such expanded evaluation would strengthen the generalizability of findings and provide practitioners with more robust guidance for integrating LLMs into enterprise-scale development processes.

CRedit authorship contribution statement

Dae-Kyoo Kim: Writing – original draft, Validation, Methodology, Investigation, Data curation, Conceptualization. **Hua Ming:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

<https://osf.io/mc96h/>.

References

- [1] A.M. Dakhel, V. Majdinasab, A. Nikanjam, F. Khomh, M.C. Desmarais, Z.M. Jiang, GitHub copilot AI pair programmer: Asset or liability? *J. Syst. Softw.* (2023) 111734.
- [2] A. Mastropaolo, L. Pascarella, E. Guglielmi, M. Ciniselli, S. Scalabrino, R. Oliveto, G. Bavota, On the robustness of code generation techniques: An empirical study on GitHub copilot, 2023, arXiv preprint [arXiv:2302.00438](https://arxiv.org/abs/2302.00438).
- [3] D.-K. Kim, J. Chen, H. Ming, L. Lu, Assessment of ChatGPT's proficiency in software development, in: *Proceedings of Congress in Computer Science, Computer Engineering, & Applied Computing*, Las Vegas, NV, 2023, pp. 2637–2644.
- [4] GitHub, Copilot, 2021, <https://copilot.github.com>.

- [5] D. Sobania, M. Briesch, F. Rothlauf, Choose your programming copilot: A comparison of the program synthesis performance of GitHub copilot and genetic programming, in: *Proceedings of the Genetic and Evolutionary Computation Conference*, Boston, MA, 2022, pp. 1019–1027.
- [6] S. Peng, E. Kalliamvakou, P. Cihon, M. Demirel, The impact of AI on developer productivity: Evidence from GitHub copilot, 2023, arXiv preprint arXiv:2302.06590.
- [7] M. Wermelinger, Using GitHub copilot to solve simple programming problems, in: *Proceedings of the 54th ACM Technical Symposium on Computing Science Education V. 1*, New York, NY, 2023, pp. 172–178.
- [8] N. Nguyen, S. Nadi, An empirical evaluation of GitHub copilot's code suggestions, in: *Proceedings of the 19th International Conference on Mining Software Repositories*, Pittsburgh, PA, 2022, pp. 1–5.
- [9] B. Yetistiren, I. Ozsoy, E. Tuzun, Assessing the quality of GitHub copilot's code generation, in: *Proceedings of the 18th International Conference on Predictive Models and Data Analytics in Software Engineering*, Singapore, Singapore, 2022, pp. 62–71.
- [10] OpenAI, ChatGPT: A language model for dialogue generation, 2022, <https://www.openai.com/chatgpt>.
- [11] Google, Gemini, 2024, <https://gemini.google.com>.
- [12] D.-K. Kim, Comparing proficiency of ChatGPT and bard in software development, in: *Generative AI for Effective Software Development*, 2024, pp. 25–51.
- [13] Anthropic, Claude: An advanced language model by Anthropic, 2023, <https://www.anthropic.com/claude>.
- [14] Meta Platforms, Inc., Meta AI: Innovations in artificial intelligence, 2023, <https://ai.meta.com>.
- [15] G. Sridhara, S. Mazumdar, et al., ChatGPT: A study on its utility for ubiquitous software engineering tasks, 2023, arXiv preprint arXiv:2305.16837.
- [16] A.I. Champa, M.F. Rabbi, C. Nachuma, M.F. Zibran, ChatGPT in action: Analyzing its use in software development, in: *Proceedings of the 21st International Conference on Mining Software Repositories*, Lisbon, Portugal, 2024, pp. 182–186.
- [17] T. Xiao, C. Treude, H. Hata, K. Matsumoto, DevGPT: Studying developer-ChatGPT conversations, in: *Proceedings of the International Conference on Mining Software Repositories*, MSR 2024, Lisbon, Portugal, 2024.
- [18] A. Nguyen-Duc, B. Cabrero-Daniel, A. Przybylek, C. Arora, D. Khanna, T. Herda, U. Rafiq, J. Melegati, E. Guerra, K.-K. Kemell, et al., Generative artificial intelligence for software engineering—A research agenda, 2023, arXiv preprint arXiv:2310.18648.
- [19] W. Rahmani, ChatGPT for software development: Opportunities and challenges, *IT Prof.* 26 (3) (2024) 80–86.
- [20] M. Waseem, T. Das, A. Ahmad, P. Liang, M. Fahmideh, T. Mikkonen, ChatGPT as a software development bot: A project-based study, in: *Proceedings of International Conference on Evaluation of Novel Approaches To Software Engineering*, Angers, France, 2024.
- [21] Y. Wang, Y. Chen, Z. Li, Z. Tang, R. Guo, X. Wang, Q. Wang, A.C. Zhou, X. Chu, Towards efficient and reliable LLM serving: A real-world workload study, 2024, arXiv preprint arXiv:2401.17644.
- [22] K.R. Felizardo, M.S. Lima, A. Deizepe, T.U. Conte, I. Steinmacher, ChatGPT application in systematic literature reviews in software engineering: An evaluation of its accuracy to support the selection activity, in: *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, Barcelona, Spain, 2024.
- [23] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, J.M. Zhang, Large language models for software engineering: Survey and open problems, in: *Proceedings of IEEE/ACM International Conference on Software Engineering: Future of Software Engineering*, Melbourne, Australia, 2023, pp. 31–53.
- [24] L. Zhong, Z. Wang, Can LLM replace stack overflow? A study on robustness and reliability of large language model code generation, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, (19) 2024, pp. 21841–21849.
- [25] R. Pudari, N.A. Ernst, From copilot to pilot: Towards AI supported software development, 2023, arXiv preprint arXiv:2303.04142.
- [26] Q. Zhang, T. Zhang, J. Zhai, C. Fang, B. Yu, W. Sun, Z. Chen, A critical review of large language model on software engineering: An example from ChatGPT and automated program repair, 2024, arXiv preprint arXiv:2310.08879.
- [27] N. Nascimento, P. Alencar, D. Cowan, Comparing software developers with ChatGPT: An empirical investigation, 2023, arXiv preprint arXiv:2305.11837.
- [28] N. Marques, R.R. Silva, J. Bernardino, Using ChatGPT in software requirements engineering: A comprehensive review, *Futur. Internet* 16 (6) (2024) 180.
- [29] A. Ahmad, M. Waseem, P. Liang, M. Fahmideh, M.S. Aktar, T. Mikkonen, Towards human-bot collaborative software architecting with ChatGPT, in: *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*, Oulu, Finland, 2023, pp. 279–285.
- [30] P. Bera, Y. Wautelet, G. Poels, On the use of ChatGPT to support agile software development, in: *Proceedings of the 2nd International Workshop on Agile Methods for Information Systems Engineering*, Zaragoza, Spain, 2023, pp. 1–9.
- [31] N. Alshahwan, M. Harman, I. Harper, A. Marginean, S. Sengupta, E. Wang, Assured offline LLM-based software engineering, in: *Proceedings of the ACM/IEEE 2nd International Workshop on Interpretability, Robustness, and Benchmarking in Neural Software Engineering*, Lisbon, Portugal, 2024, pp. 7–12.
- [32] A. Rajbhoj, A. Somase, P. Kulkarni, V. Kulkarni, Accelerating software development using generative AI: Chatgpt case study, in: *Proceedings of the 17th Innovations in Software Engineering Conference*, Bangalore, India, 2024, pp. 1–11.
- [33] I. Solohubov, A. Moroz, M.Y. Tiahunova, H.H. Kyrychek, S. Skrupsky, Accelerating software development with AI: Exploring the impact of ChatGPT and GitHub copilot, in: *Proceedings of the 11th Workshop on Cloud Technologies in Education*, Kryvyi Rih, Ukraine, 2023, pp. 76–86.
- [34] Z. Özpölat, Ö. Yıldırım, M. Karabatak, Artificial intelligence-based tools in software development processes: Application of ChatGPT, *Eur. J. Tech. (EJT)* 13 (2) (2023) 229–240.
- [35] A.E. Hassan, G.A. Oliva, D. Lin, B. Chen, Z. Ming, et al., Rethinking software engineering in the foundation model era: From task-driven AI copilots to goal-driven AI pair programmers, 2024, arXiv preprint arXiv:2404.10225.
- [36] J. White, S. Hays, Q. Fu, J. Spencer-Smith, D.C. Schmidt, ChatGPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design, 2023, arXiv preprint arXiv:2303.07839.
- [37] C. Larman, “Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development”, third ed., Prentice Hall, 2004.
- [38] M.R. Lyu, *Handbook of Software Reliability Engineering*, vol. 222, IEEE Computer Society Press, Los Alamitos, CA, 1996.
- [39] S. Xu, Y. Peng, Z. Xu, Z. Lu, Reasoning before comparison: LLM-enhanced semantic similarity metrics for domain specialized text analysis, 2024, arXiv preprint arXiv:2402.11398.
- [40] S. Herbold, Semantic similarity prediction is better than other semantic similarity measures, *Trans. Mach. Learn. Res.* (2024).
- [41] D.-K. Kim, Reliability and similarity, 2024, <https://osf.io/mc96h/>.